

MoveInsight: Herramienta para el análisis de patrones de movimiento y prevención de lesiones

**MoveInsight: Tool for motion pattern analysis
and injury prevention**

Trabajo de Fin de Grado

Grado en Ingeniería Informática



**VNiVERSiDAD
D SALAMANCA**

Junio de 2026

Autor

Diego Gómez Terradillos

Tutora

Dña. Carolina Zato Domínguez

Certificado del tutor TFG

D./Dña. **D. Carolina Zato Domínguez**, profesor/a del Departamento de **Informática y Automática** de la Universidad de Salamanca,

HACE CONSTAR:

Que el trabajo titulado “**MoveInsight: Herramienta para el análisis de patrones de movimiento y prevención de lesiones**”, que se presenta, ha sido realizado por **Diego Gómez Terradillos**, con DNI ******9324C** y constituye la memoria del trabajo realizado para la superación de la asignatura Trabajo de Fin de Grado en Ingeniería Informática en esta Universidad.

Salamanca, 20 de junio de 2026

Fdo.: D. Carolina Zato Domínguez

Resumen

El análisis biomecánico del movimiento humano constituye una herramienta fundamental en la rehabilitación deportiva y la prevención de lesiones. Sin embargo, el acceso a sistemas de evaluación precisos y automatizados sigue siendo limitado fuera de entornos clínicos especializados. Este Trabajo Fin de Grado presenta MoveInsight, una aplicación nativa Android desarrollada en Kotlin con Jetpack Compose que, apoyada en un backend desarrollado en FastAPI, automatiza el análisis de la técnica de sentadilla mediante visión por ordenador e inteligencia artificial.

El usuario graba o sube un vídeo de su ejecución y obtiene una evaluación de los principales indicadores biomecánicos: profundidad, inclinación del torso, valgo de rodilla, simetría y ritmo. El procesamiento parte de la estimación de pose tridimensional con MediaPipe (BlazePose), a partir de la cual se calculan las variables biomecánicas que alimentan dos modelos complementarios: una red neuronal recurrente BiLSTM, encargada de segmentar las fases de la sentadilla y delimitar las repeticiones, y un modelo XGBoost, que evalúa la calidad de cada repetición clasificando cada indicador en tres niveles (erróneo, aceptable y excelente). Ambos modelos se entrenaron sobre un conjunto de datos propio, construido y etiquetado específicamente durante el desarrollo del proyecto, y se refinaron a lo largo de sucesivas versiones.

La arquitectura integra el procesamiento del vídeo, la extracción de los puntos clave corporales, la inferencia de los modelos y la generación de informes, que el usuario consulta en la aplicación. Esta incorpora, además, un módulo de seguimiento longitudinal del estado físico y del bienestar, con recordatorios y resúmenes gestionados mediante WorkManager. El resultado es una herramienta funcional, accesible y de bajo coste que acerca el análisis biomecánico al usuario final.

Palabras Clave: Biomecánica, sentadilla, análisis de movimiento, visión por ordenador, inteligencia artificial, BiLSTM, XGBoost, MediaPipe, Android, Jetpack Compose, FastAPI.

Abstract

Biomechanical analysis of human movement is a fundamental tool in sports rehabilitation and injury prevention. However, access to accurate, automated assessment systems remains limited outside specialised clinical settings. This Bachelor's Thesis presents MoveInsight, a native Android application developed in Kotlin with Jetpack Compose which, supported by a FastAPI backend, automates the analysis of squat technique using computer vision and artificial intelligence.

The user records or uploads a video of their execution and obtains an assessment of the main biomechanical indicators: depth, torso inclination, knee valgus, symmetry and rhythm. Processing begins with three-dimensional pose estimation using MediaPipe (BlazePose), from which the biomechanical variables that feed two complementary models are computed: a BiLSTM recurrent neural network, responsible for segmenting the squat phases and delimiting the repetitions, and an XGBoost model that assesses the quality of each repetition by classifying each indicator into three levels (incorrect, acceptable and excellent). Both models were trained on a custom dataset, built and labelled specifically during the project, and refined across successive versions.

The architecture integrates video processing, body keypoint extraction, model inference and report generation, which the user consults within the application. It also includes a longitudinal tracking module for physical readiness and well-being, with reminders and summaries managed through WorkManager. The result is a functional, accessible and low-cost tool that brings biomechanical analysis closer to the end user.

Keywords: Biomechanics, squat, motion analysis, computer vision, artificial intelligence, BiLSTM, XGBoost, MediaPipe, Android, Jetpack Compose, FastAPI.

Agradecimientos

Antes de comenzar, quiero dedicar estas primeras palabras a quienes me han acompañado y ayudado a lo largo de este camino. Sin ellos, este trabajo no habría sido posible.

A mis padres, por estar siempre a mi lado y sostenerme en cada paso, por su apoyo incondicional y por confiar en mí incluso cuando yo era el primero en dudar. Les debo todo lo que soy, y nada de esto habría sido posible sin ellos.

A mi tutora, Carolina, por implicarse de lleno en este proyecto desde el primer día, por confiar en mí y por estar ahí en todo aquello en lo que he necesitado su ayuda. Su orientación y su cercanía han sido fundamentales para llegar hasta aquí.

A la Universidad de Salamanca y a todos los profesores que me han acompañado a lo largo de estos años, por transmitirme sus conocimientos con dedicación y por contribuir, cada uno a su manera, a la formación que hoy me permite cerrar esta etapa. Me llevo mucho más que lo aprendido en las aulas.

A mis amigos de la universidad, por hacer mucho más amenos estos años de carrera, por las risas, por las horas compartidas y por convertir el día a día en algo que recordaré con cariño. Y a mis amigos de toda la vida, por seguir ahí pase lo que pase, por ayudarme a desconectar siempre que nos vemos y por echarnos siempre unas risas juntos.

A Sara, por estar siempre a mi lado y apoyarme en todo momento, también en los días difíciles. Gracias por tu paciencia, por animarme cuando más lo necesitaba y por estar ahí en cada paso de este camino, haciendo que todo fuera más fácil a tu lado.

Por último, quiero dar las gracias a quienes contribuyeron directamente a este trabajo. A Juan Andrés García Soria, graduado en Ciencias de la Actividad Física y del Deporte (CAFYD), por compartir conmigo sus conocimientos de biomecánica, que motivaron la incorporación del centro de sustentación como variable de análisis, y por ser, además de un apoyo en este proyecto, un gran amigo que siempre está para todo lo que necesito. Y a Marcos Arias González, de BISITE, por sus consejos para la implementación del modelo XGBoost, por resolver siempre mis dudas y por la rapidez y la buena disposición con las que me ayudó en cada momento.

A todos, gracias de corazón.

Índice de Contenidos

Certificado del tutor TFG	i
Resumen	ii
Abstract.....	iii
Agradecimientos.....	iv
Índice de Contenidos	v
Índice de Tablas.....	vii
Índice de Figuras	vii
1 Introducción	1
1.1 Estado del arte	3
2 Objetivos del proyecto	5
2.1 Objetivo principal	5
2.2 Subobjetivos funcionales	6
2.3 Objetivos técnicos y de aprendizaje personal.....	7
3 Conceptos teóricos	9
3.1 Preprocesado con OpenCV	9
3.2 Visión por ordenador y estimación de pose	10
3.3 Aprendizaje automático	13
3.4 Redes neuronales recurrentes (RNN, LSTM y BiLSTM)	14
3.5 Modelos de árboles y gradient boosting (XGBoost)	16
4 Técnicas y herramientas.....	18
4.1 Metodología de desarrollo	18
4.2 Aproximaciones al reconocimiento del movimiento.....	19
4.3 Visión por ordenador: OpenCV y MediaPipe BlazePose.....	19
4.4 Modelos de aprendizaje automático	20
4.5 Fuentes de datos para el entrenamiento	21

4.6	Métricas de evaluación	22
4.7	Backend y API REST	22
4.8	Aplicación móvil	22
4.9	Infraestructura y despliegue.....	22
4.10	Herramientas de desarrollo y gestión del proyecto	23
5	Aspectos Relevantes del Desarrollo	24
5.1	Visión general de la arquitectura del sistema	24
5.2	Pipeline de visión artificial	25
5.3	Motor de biomecánica	27
5.4	Construcción del dataset.....	38
5.5	Diseño y evolución de los modelos	41
5.6	Backend y procesamiento asíncrono	47
5.7	Aplicación móvil	50
5.8	Seguimiento del bienestar y prevención de lesiones	54
5.9	Despliegue	59
6	Limitaciones, líneas de trabajo futuras y conclusiones.....	62
6.1	Limitaciones	62
6.2	Líneas de trabajo futuras.....	63
6.3	Conclusión técnica.....	65
6.4	Conclusión personal	66
	Lista de acrónimos.....	68
	Glosario	70
	Bibliografía.....	72

Índice de Tablas

Tabla 1. Comparativa con las apps de la competencia	4
Tabla 2. Resumen de filtros, suavizado y normalización en el sistema	31
Tabla 3. Descripción de los distintos modelos comparados	41
Tabla 4. Evolución de las versiones del modelo BiLSTM.....	42
Tabla 5. Resultados de la evaluación de predicción de fase.....	46
Tabla 6. Resultados de la evaluación de KPIs.....	46
Tabla 7. Contenedores Docker	59

Índice de Figuras

Figura 1. Keypoints extraídos por MediaPipe	12
Figura 2. Arquitectura del sistema.....	24
Figura 3. Proceso de suavizado	29
Figura 4. Descripción de las métricas utilizadas en ambas redes neuronales	33
Figura 5. Detección del perfil de grabación	37
Figura 6. Arquitectura de dos etapas: BiLSTM + XGBoost	44
Figura 7. Flujo asíncrono en el servidor	49
Figura 8. Cálculo del semáforo	56
Figura 9. Motor de recomendaciones	57

1 Introducción

En los últimos años, el auge del entrenamiento autónomo en gimnasios sin supervisión o directamente en casa ha multiplicado el número de personas que practican ejercicios de fuerza. La sentadilla, más concretamente es uno de los ejercicios fundamentales del entrenamiento de fuerza, un ejercicio entrenado por una gran cantidad de personas de diferentes edades, tanto chicos jóvenes como personas mayores, desde sin peso a pesos extremos en competiciones de powerlifting. Por su carácter multiarticular y su transferencia a multitud de gestos cotidianos y deportivos, está presente tanto en programas de rendimiento como en procesos de rehabilitación y de acondicionamiento físico general.

Esta popularización tiene una contrapartida. La sentadilla es un ejercicio técnicamente exigente, y una ejecución incorrecta, no solo reduce su eficacia, sino que incrementa el riesgo de lesión, sin contar con el riesgo extra si utilizamos pesos demandantes. La valoración fiable de la técnica al ejecutar el movimiento ha requerido tradicionalmente la presencia de un entrenador o un fisioterapeuta, o bien el uso de equipamiento de laboratorio costoso y poco accesible.

El resultado es una brecha: la mayoría de los usuarios carece de una forma objetiva, inmediata y asequible de saber si está ejecutando correctamente el movimiento, por lo que mucha gente opta por no cargar peso para reducir el riesgo de lesión o simplemente por dejar de realizar un ejercicio con tantos beneficios como es este.

Los avances recientes en visión por ordenador y aprendizaje automático han hecho posible algo impensable hace pocos años: estimar la postura tridimensional del cuerpo humano a partir del vídeo de una única cámara convencional, como la de cualquier teléfono móvil. Esta posibilidad de tener acceso a la inteligencia artificial mediante un dispositivo que prácticamente todo el mundo lleva en el bolsillo abre la puerta a democratizar el análisis biomecánico y acercarlo al usuario final, sin necesidad de sensores, marcadores ni instalaciones especializadas. La motivación de este trabajo nace precisamente de esa oportunidad, unida al interés personal por aplicar técnicas de inteligencia artificial, visión por ordenador, desarrollo y entrenamiento de modelos y a un problema real y tangible, que derive en un uso de la tecnología con un cierto valor para la sociedad.

En este contexto se enmarca el presente Trabajo Fin de Grado, que propone MoveInsight: un sistema capaz de analizar de forma automática la técnica de la sentadilla a partir de un simple vídeo grabado con un teléfono Android.

El sistema extrae la pose del usuario, evalúa mediante modelos de inteligencia artificial los principales indicadores biomecánicos de cada repetición y devuelve una valoración comprensible, acompañada de recomendaciones y de un seguimiento de su evolución a lo largo del tiempo. Conviene subrayar desde el inicio que MoveInsight se concibe como una herramienta de apoyo y en ningún caso pretende sustituir el criterio de un profesional médico o deportivo.

El resto de esta memoria se organiza de la siguiente forma: el capítulo 2 detalla los objetivos del proyecto, tanto funcionales como de aprendizaje personal. El capítulo 3 introduce los conceptos teóricos necesarios para comprender el trabajo y el capítulo 4 describe las técnicas y herramientas empleadas. El capítulo 5 expone los aspectos más relevantes del desarrollo, el capítulo 6 recoge las conclusiones alcanzadas y las líneas de trabajo futuro, y para finalizar, se incluyen una lista de los acrónimos utilizados en la memoria, un glosario que explica los términos más técnicos y una bibliografía con las fuentes consultadas. La documentación se completa con una serie de anexos, los cuales tratarán los siguientes temas y desarrollan en detalle los aspectos más técnicos del proyecto:

- Anexo I. Especificaciones del sistema.
- Anexo II. Análisis y diseño del sistema.
- Anexo III. Estimación del tamaño y del esfuerzo.
- Anexo IV. Plan de seguridad.
- Anexo V. Manual de usuario.
- Anexo VI. Manual del programador.

1.1 Estado del arte

Antes de comenzar el trabajo se revisaron las aplicaciones disponibles en el mercado relacionadas con el entrenamiento de fuerza, con el fin de determinar si había alguna aplicación similar o por el contrario había una brecha en la que nuestra aplicación pudiera introducirse. Esta serie de aplicaciones observadas pueden agruparse en varias categorías, pero ninguna de ellas cubre el objetivo de MoveInsight.

Los registros de entrenamiento, como Hevy o RepCount, funcionan a modo de “libreta digital”, permiten anotar series, repeticiones y peso, e incluso registrar el esfuerzo percibido, pero todo el ingreso de datos es manual y no validan la realización del ejercicio del usuario. Registran cuánto peso dice haber movido el usuario, pero no cómo ha ejecutado el movimiento ni evalúan su técnica.

Las aplicaciones con inteligencia artificial generativa, como Fitbod o Fitify, emplean algoritmos para crear rutinas personalizadas y muestran vídeos demostrativos de cómo realizar los ejercicios, pero no observan al usuario, es decir, ofrecen un feedback unidireccional. Su inteligencia artificial es predictiva (es decir, de planificación), no de visión por ordenador, por lo que no analiza la ejecución real. Son aplicaciones simples de planificación.

Las aplicaciones de retos gamificados (del tipo «100, 200 o 300 sentadillas») fomentan el volumen progresivo mediante cronómetros, y usando técnicas de progresiones generales para este tipo de ejercicios, sin ningún control de la técnica, lo que puede incrementar el riesgo de lesión, o las diferencias entre cada persona y el distinto volumen de trabajo que cada persona distinta acepta.

Por otra parte, algunas aplicaciones de conteo basadas en los sensores del móvil en este caso el acelerómetro automatiza el recuento de repeticiones, pero son “ciegas” a la postura: detectan movimiento, no si se alcanza la profundidad adecuada o si las rodillas colapsan hacia dentro, y resultan fácilmente falseables. Además de obtener un resultado poco preciso en este conteo. Las únicas aplicaciones detectadas que cuentan repeticiones lo hacen mediante el acelerómetro del teléfono, lo cual hace que el resultado además sea bastante impreciso, particularmente en ejercicios de sentadillas donde el ritmo es variable.

Por último, en los últimos años ha surgido una categoría de aplicaciones que sí analizan la ejecución del ejercicio mediante visión por ordenador y estimación de pose. En el ámbito de la fisioterapia y la rehabilitación, soluciones como Kaia Health, Exer,

Sency o Kemtai guían al usuario y evalúan determinados movimientos a partir de la cámara; en el del análisis técnico-deportivo, herramientas como Onform o Coach's Eye permiten revisar la ejecución sobre el vídeo. Son la referencia más cercana a MoveInsight, pero presentan diferencias sustanciales: la mayoría se orientan a la rehabilitación genérica o al análisis manual del vídeo, trabajan habitualmente con coordenadas bidimensionales y no realizan un análisis biomecánico específico de la sentadilla de fuerza con indicadores definidos (profundidad, valgo de rodilla, inclinación del torso, simetría y ritmo). Ninguna integra, además, el análisis de la técnica con un seguimiento longitudinal del estado físico, las molestias y la carga de entrenamiento.

De este análisis se concluye que, si bien existen aplicaciones que abordan aspectos aislados como el registro del entrenamiento, la planificación, el conteo o incluso el análisis del movimiento por visión, ninguna cubre la intersección que persigue MoveInsight: un análisis biomecánico específico de la sentadilla, que infiere la postura tridimensional a partir de una única cámara convencional, combinado con un seguimiento estructurado de la evolución del usuario, sus molestias y su carga de entrenamiento. Es precisamente en ese hueco donde se sitúa este trabajo.

Característica / Tecnología	MoveInsight	Hevy / RepCount	Fitbod / Fitify	Apps de Retos (0-200)	Squats	Apps de técnica por visión
Ingreso de Repeticiones	Automático (IA / Vídeo)	Manual	Manual	Manual	Automático (Sensor móvil)	Automático (IA / Vídeo)
Evaluación de Técnica	Sí (Visión Artificial)	No	No	No	No (Solo detecta movimiento)	Parcial (genérica, 2D)
Métrica RPE (Esfuerzo)	Sí	Sí (En avanzadas)	No	No	No	No
Seguimiento de Lesiones / Dolor	Sí (Específico)	No	No (Solo % fatiga)	No	No	Parcial (rehabilitación)

Tabla 1. Comparativa con las apps de la competencia

Revisando el tipo de apps alternativas, y el estado del arte en este entorno, es donde MoveInsight gana valor, al ser una aplicación con técnicas novedosas, y que permite al usuario hacer un seguimiento completo, con recomendaciones y consejos, pero siempre con el límite marcado de que la aplicación no es profesional médico ni deportivo.

2 Objetivos del proyecto

2.1 Objetivo principal

El objetivo principal de MoveInsight es poner el análisis biomecánico del entrenamiento de fuerza al alcance de cualquier usuario, ofreciendo un sistema de seguimiento completo de sus sesiones de sentadilla a través de un dispositivo móvil convencional. Como ya se ha mencionado anteriormente, cabe destacar que MoveInsight no pretende en ningún caso sustituir el criterio de un médico, un entrenador personal o un fisioterapeuta, sus evaluaciones son indicadores automatizados de carácter general, cuya interpretación debe siempre contextualizarse en función de las características individuales de cada usuario.

Con seguimiento completo se entiende un conjunto integrado de funcionalidades que abarca desde el análisis técnico de cada repetición hasta la gestión del estado físico a lo largo del tiempo. Tras grabar o subir un vídeo desde la aplicación, el sistema extrae automáticamente el número de repeticiones realizadas, evalúa los principales indicadores biomecánicos de cada una: profundidad, inclinación del torso, valgo de rodilla, simetría y ritmo; y genera un informe con las puntuaciones obtenidas, su comparación respecto a sesiones anteriores y un indicador de fatiga a lo largo de la propia sesión. Como apoyo visual, la aplicación reconstruye además un vídeo con el esqueleto del usuario superpuesto sobre su ejecución, lo que permite revisar ejecuciones antiguas y comprobar si ha habido mejora. Además, el usuario puede exportar el informe completo y detallado de toda su información en formato PDF. La aplicación y también incorpora un componente de mejora técnica mediante sugerencias personalizadas y material audiovisual explicativo que el usuario puede consultar para corregir los errores detectados. Por último, el sistema incluye un módulo de gestión de la carga de entrenamiento que, a partir del volumen acumulado de sesiones y del estado físico registrado por el propio usuario, es capaz de detectar señales de desgaste y recomendar periodos de descanso, o bien indicar cuándo el nivel de actividad es insuficiente para mantener la progresión.

Todo este historial es consultable en la aplicación a través de gráficas de evolución que permiten visualizar la progresión de cada indicador a lo largo del tiempo. El usuario puede además registrar lesiones o molestias activas, de forma que el sistema pueda tenerlas en cuenta como contexto adicional en la interpretación de los resultados.

La accesibilidad es un principio transversal del sistema: el único requisito para el usuario es disponer de un dispositivo Android con cámara con el que grabarse y la aplicación instalada, sin necesidad de equipamiento adicional costoso ni conocimientos técnicos previos, ni disponer de un entrenador.

2.2 Subobjetivos funcionales

El catálogo formal de objetivos del sistema (OBJ-01 a OBJ-07), los casos de uso y los requisitos funcionales, de información y no funcionales se detallan en el Anexo I (Especificaciones del sistema). A continuación, se enumeran los subobjetivos funcionales del proyecto, que describen las capacidades concretas que el sistema debe satisfacer.

- Extraer la pose corporal del vídeo: implementar un pipeline que obtenga las coordenadas tridimensionales de los puntos clave del cuerpo, fotograma a fotograma, mediante MediaPipe BlazePose.
- Calcular las métricas biomecánicas: desarrollar un motor que, a partir de las coordenadas, obtenga las métricas articulares, de estabilidad y de ritmo, aplicando filtrado, suavizado y normalización.
- Segmentar las fases de la sentadilla: entrenar una red BiLSTM que clasifique cada fotograma en su fase (de pie, bajando, abajo y subiendo) y permita delimitar y contar las repeticiones.
- Evaluar los KPIs por repetición: entrenar un modelo XGBoost que clasifique cada indicador (profundidad, torso, valgo, simetría y ritmo) en las categorías erróneo, aceptable y excelente.
- Detectar el perfil de grabación: determinar automáticamente si la toma es frontal o lateral para adaptar los indicadores evaluables a cada plano.
- Construir un conjunto de datos propio: recopilar y grabar vídeos de sentadillas, desarrollar una herramienta de etiquetado y establecer una metodología para ampliar el dataset de forma continua.
- Evaluar el rendimiento de los modelos: medir la segmentación de fases (precisión, recall y F1 por clase) y los KPIs (validación cruzada agrupada por vídeo) sobre datos no usados en el entrenamiento.

- Desarrollar el backend y la API REST: construir con FastAPI una API que gestione la recepción de vídeos, la ejecución del análisis, la autenticación y el almacenamiento de resultados.
- Desarrollar la aplicación Android: crear una aplicación con arquitectura limpia y patrón MVVM para grabar o subir vídeos, consultar resultados y visualizar el historial.
- Implementar la autenticación de usuarios: gestionar el acceso mediante JWT con tokens de acceso y refresco, renovados de forma transparente.
- Ofrecer seguimiento longitudinal del usuario: registrar la evolución de los indicadores y las molestias o lesiones, y gestionar la carga de entrenamiento con notificaciones locales.
- Implementar una función de descarga del contenido en PDF, donde se pueda recoger todas las sesiones, diagramas, consejos, datos, ...
- Proporcionar contenido educativo: incorporar recomendaciones personalizadas y material audiovisual asociados a los errores detectados.
- Desplegar el sistema en producción: desplegar el conjunto con Docker y docker-compose, con Nginx como proxy inverso y HTTPS.

En conjunto, estos subobjetivos cubren el ciclo completo del sistema, desde la captura del vídeo hasta la evaluación de la técnica y el seguimiento del usuario a lo largo del tiempo.

2.3 Objetivos técnicos y de aprendizaje personal

Más allá de los requisitos funcionales, este proyecto persigue una serie de objetivos de carácter técnico y de aprendizaje personal que han orientado las decisiones de diseño a lo largo del desarrollo. Estos se enumeran a continuación:

- Aprender a investigar y estudiar el estado del arte: documentarse con rigor leyendo trabajos previos y comparando técnicas antes de diseñar, para fundamentar las decisiones.
- Aplicar técnicas de visión por computador a un problema real: trabajar con datos propios recogidos en condiciones no controladas (iluminación, encuadre y dispositivos diversos).
- Completar el ciclo de un proyecto de aprendizaje automático: recopilar y etiquetar datos, entrenar, evaluar e integrar los modelos en producción.

- Profundizar en el desarrollo Android moderno: construir una aplicación con Jetpack Compose y arquitectura limpia, más allá de los ejemplos académicos.
- Diseñar una API REST de producción: implementar con FastAPI un backend que gestione autenticación, persistencia y procesamiento asíncrono.
- Aplicar buenas prácticas de ingeniería del software: emplear separación de responsabilidades, inyección de dependencias y patrones de diseño en un proyecto de alcance real.
- Desplegar y operar un sistema en producción: afrontar los problemas reales del despliegue, como introducir todo en un contenedor, el HTTPS, la gestión de recursos y la compatibilidad de hardware.
- Experimentar el ciclo completo de un proyecto real: abarcar todas las fases de un sistema de principio a fin, frente a los ejercicios académicos acotados y autocontenidos.

Estos objetivos recogen las competencias técnicas y transversales que se han buscado adquirir durante el desarrollo del trabajo.

3 Conceptos teóricos

3.1 Preprocesado con OpenCV

OpenCV (Open Source Computer Vision Library) es una biblioteca de visión por ordenador para el tratamiento de imágenes y vídeo, es de código abierto y es realmente de referencia. En este sistema se utiliza en la fase de preprocesado: el conjunto de operaciones que preparan cada fotograma del vídeo antes de que el modelo de estimación de pose lo analice.

El preprocesado persigue adaptar el formato de los datos al que esperan las herramientas posteriores y preparar el vídeo para su recorrido temporal. A continuación, se describen las operaciones que lo componen y el motivo de cada una.

3.1.1 El procesado fotograma a fotograma

Un vídeo digital es una sucesión de imágenes fijas, llamadas fotogramas (frames), mostradas a gran velocidad. Cada fotograma es, a su vez, una matriz de píxeles, donde cada píxel almacena un valor de color. El análisis se realiza fotograma a fotograma: el sistema recorre el vídeo extrayendo cada imagen de forma secuencial y aplicando sobre ella la estimación de pose.

El motivo de este recorrido secuencial es que la postura del cuerpo cambia en cada instante; solo procesando cada fotograma por separado se obtiene la evolución temporal del movimiento, de la que dependen magnitudes como las velocidades, el ritmo o la segmentación de las fases. OpenCV proporciona el mecanismo para abrir el vídeo y leer sus fotogramas uno a uno, además de informar de la cadencia (fotogramas por segundo, fps), un dato imprescindible para situar correctamente los cálculos en el tiempo.

3.1.2 Cambio de espacio de color (BGR $\rightarrow$ RGB)

Un espacio de color define cómo se representa numéricamente el color de cada píxel. El más habitual es el RGB, que descompone cada píxel en tres componentes: rojo (Red), verde (Green) y azul (Blue). Por motivos históricos, OpenCV carga las imágenes en el orden inverso, BGR (azul, verde, rojo), mientras que la mayoría de las herramientas de visión, entre ellas el modelo de estimación de pose empleado, esperan la imagen en orden RGB.

Por ello es necesario realizar una conversión de espacio de color de BGR a RGB antes de entregar el fotograma al modelo. Omitir este paso no provoca un fallo de ejecución, pero sí los resultados podrían verse ligeramente alterados, pudiendo derivar en resultados incorrectos, ya que el modelo interpretaría los canales de color intercambiados.

3.1.3 Resolución de los fotogramas

La resolución de un fotograma es el número de píxeles que lo componen. Una técnica habitual de preprocesado en visión por computador es el redimensionado (resize): reducir la resolución de la imagen para disminuir el coste de cómputo, ya que el tiempo de procesado crece con el número de píxeles. En este sistema, sin embargo, no se aplica un redimensionado manual, y los fotogramas se procesan a su resolución original.

Hay dos grandes razones para esto. En primer lugar, el modelo de estimación de pose realiza internamente su propio escalado: adapta cada imagen al tamaño de entrada que necesita la red, por lo que un redimensionado previo por parte de la aplicación sería redundante e incluso podría descartar información antes de que el modelo aplique su escalado óptimo. En segundo lugar, el análisis se ejecuta en el servidor de forma asíncrona, y no en tiempo real sobre el dispositivo, por lo que el coste de procesar a resolución completa es asumible y se prioriza la precisión de la detección frente a la velocidad.

3.1.4 Renderizado de la salida

Aunque no forma parte estricta del preprocesado de la entrada, OpenCV se emplea también en sentido inverso: para dibujar (renderizar) sobre los fotogramas originales el esqueleto detectado, generando el vídeo con el esqueleto superpuesto que se devuelve al usuario. Para ello se utilizan las funciones de dibujo de la biblioteca (líneas y círculos) aplicadas sobre cada fotograma. El resultado es el vídeo de entrada con los 33 puntos extraídos por MediaPipe dibujados en el vídeo original, para que el usuario pueda observarlo.

3.2 Visión por ordenador y estimación de pose

3.2.1 Visión por ordenador

La visión por ordenador es la rama de la inteligencia artificial que dota a los ordenadores de la capacidad de interpretar y extraer información a partir de imágenes y vídeos, de forma análoga a como lo haría el sistema visual humano. Sus tareas van desde

la clasificación de imágenes o la detección de objetos hasta la segmentación o el análisis del movimiento. En este proyecto, la visión por ordenador es la tecnología que permite extraer la postura del cuerpo del usuario a partir del vídeo de la sentadilla, que constituye el punto de partida de todo el análisis posterior.

3.2.2 *Deep learning*

La técnica empleada para esa extracción se basa en el deep learning, por lo que conviene definirlo. El deep learning es un subcampo del aprendizaje automático (machine learning), que a su vez agrupa los algoritmos capaces de aprender patrones a partir de datos en lugar de seguir reglas programadas de forma explícita. En concreto, el deep learning emplea redes neuronales artificiales con múltiples capas (de ahí lo de "profundo"), inspiradas de forma simplificada en la organización de las neuronas del cerebro.

Su principal ventaja frente a los métodos clásicos es que aprende automáticamente las características relevantes directamente de los datos en bruto (por ejemplo, los píxeles de una imagen), sin necesidad de que un programador defina manualmente qué buscar. Esta capacidad resulta determinante en problemas complejos y variables como el cuerpo humano, articulado y deformable, donde definir reglas a mano sería inviable. El modelo de estimación de pose utilizado es, precisamente, una red neuronal profunda entrenada sobre grandes conjuntos de imágenes de personas etiquetadas.

3.2.3 *Estimación de pose*

La estimación de pose es la tarea de visión por ordenador que consiste en detectar la postura del cuerpo humano en una imagen o vídeo, localizando un conjunto de puntos anatómicos predefinidos. Su resultado son las coordenadas de esos puntos en cada fotograma, que describen cómo está colocado el cuerpo en ese instante. Es la base sobre la que se construye cualquier análisis del movimiento, ya que traduce la imagen, difícil de tratar directamente, en una representación numérica y manejable de la postura.

3.2.4 *Keypoints corporales*

Los puntos anatómicos que localiza la estimación de pose se denominan puntos clave o keypoints en inglés. Se corresponden con las articulaciones y referencias del cuerpo: hombros, codos, muñecas, caderas, rodillas, tobillos, etc. El modelo empleado detecta 33 landmarks por persona. Cada uno lleva asociadas sus coordenadas y un valor de

visibilidad o fiabilidad, que indica la confianza del modelo en esa detección y que resulta útil para descartar puntos poco fiables.

El conjunto de todos los landmarks conectados forma el esqueleto que representa la postura del usuario. En la siguiente imagen se pueden observar los 33 keypoints extraídos:

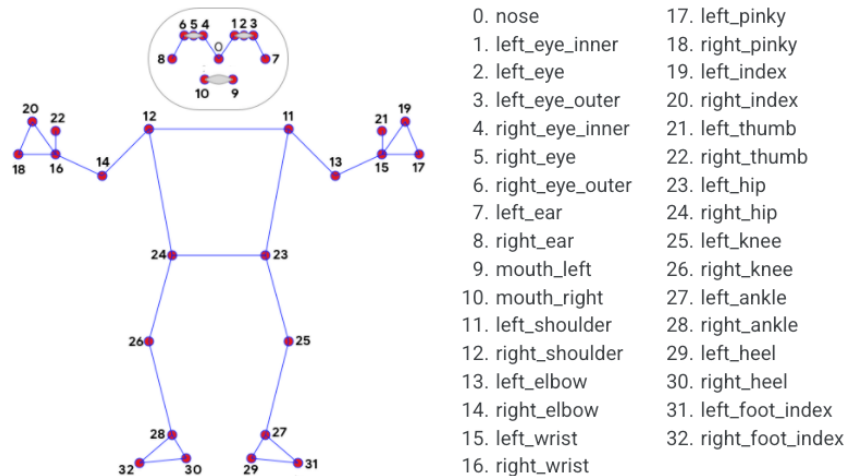


Figura 1. Keypoints extraídos por MediaPipe

3.2.5 Coordenadas 2D y 3D

Cada keypoint se expresa mediante coordenadas. En dos dimensiones (2D), un punto se describe con un par de valores (x, y) que indican su posición en el plano de la imagen (horizontal y vertical). En tres dimensiones (3D) se añade una tercera coordenada, z, que representa la profundidad (la distancia respecto a la cámara), situando el punto en el espacio.

La diferencia es relevante para este trabajo. Una representación puramente 2D pierde la información de profundidad, lo que distorsiona el cálculo de ángulos articulares cuando el cuerpo no está perfectamente orientado hacia la cámara; en un movimiento tridimensional como la sentadilla, esto introduce errores. Por el contrario, disponer de coordenadas 3D permite calcular esos ángulos con mayor fidelidad. El modelo de estimación de pose utilizado estima las tres coordenadas (x, y, z) a partir de una única cámara, lo que constituye una de las razones de su elección. Conviene precisar, no obstante, que la coordenada de profundidad se infiere (no se mide directamente), por lo que es menos precisa que las coordenadas del plano de la imagen, una limitación que el sistema debe tener en cuenta.

3.3 Aprendizaje automático

El aprendizaje automático (machine learning) es la disciplina de la inteligencia artificial que estudia los algoritmos capaces de aprender a realizar una tarea a partir de ejemplos, en lugar de mediante reglas programadas de forma explícita. En vez de que un programador codifique paso a paso cómo resolver el problema, se le proporcionan al modelo datos de los que extrae por sí mismo los patrones que relacionan una entrada con la salida deseada. Una vez aprendidos esos patrones, el modelo puede aplicarlos a datos nuevos. A continuación, se introducen los conceptos del aprendizaje automático relevantes para este trabajo.

3.3.1 Aprendizaje supervisado

Dentro del aprendizaje automático, el aprendizaje supervisado es el paradigma en el que el modelo aprende a partir de ejemplos etiquetados: cada dato de entrada va acompañado de su salida correcta (su etiqueta). El modelo ajusta sus parámetros para reproducir esa correspondencia y, una vez entrenado, es capaz de predecir la salida de datos no vistos. Se contrapone al aprendizaje no supervisado, en el que los datos carecen de etiqueta y el modelo busca estructura por sí mismo.

Los dos modelos de este sistema son supervisados: tanto la red BiLSTM como el modelo XGBoost se entrenan con repeticiones de sentadilla cuyas fases y KPIs han sido etiquetados manualmente, de modo que aprenden a reproducir esos criterios sobre ejecuciones nuevas.

3.3.2 Clasificación

Las tareas de aprendizaje supervisado se dividen, según la naturaleza de la salida, en clasificación y regresión. En la clasificación, la salida es una categoría de un conjunto finito (por ejemplo: bajando, dentro del conjunto de: de pie, bajando, abajo y subiendo); en la regresión, la salida es un valor numérico continuo.

Este proyecto aborda problemas de clasificación: la BiLSTM clasifica cada fotograma en una de las cuatro fases de la sentadilla (de pie, bajando, abajo y subiendo), y el modelo XGBoost clasifica cada indicador en una de tres categorías (erróneo, aceptable y excelente).

3.3.3 Entrenamiento y validación

Construir un modelo supervisado parte de dividir los datos disponibles en subconjuntos independientes. Con el conjunto de entrenamiento, el modelo ajusta sus parámetros para minimizar el error sobre esos ejemplos. Con un conjunto de validación, el cual son datos que el modelo no ha visto durante el entrenamiento, se evalúa su capacidad de generalizar, es decir, de funcionar correctamente ante datos nuevos, que es el objetivo real del sistema.

Esta separación es fundamental: medir el rendimiento sobre los mismos datos con los que se ha entrenado daría una estimación engañosamente optimista, ya que el modelo podría limitarse a reproducir ejemplos que ya conoce en lugar de demostrar que ha aprendido el patrón general.

3.3.4 Sobreajuste (overfitting)

El sobreajuste se produce cuando un modelo aprende los datos de entrenamiento demasiado bien, memorizando incluso su ruido y sus particularidades, en lugar de capturar el patrón general subyacente. Como consecuencia, acierta sobre los datos de entrenamiento, pero falla con datos nuevos: ha perdido capacidad de generalización. El fenómeno opuesto, el subajuste (underfitting), ocurre cuando el modelo es demasiado simple para capturar el patrón.

El sobreajuste se detecta por la diferencia de rendimiento entre el conjunto de entrenamiento y el de validación: si el modelo acierta mucho en el primero y poco en el segundo, está sobreajustando. Es un riesgo especialmente acusado cuando se dispone de pocos datos y modelos de gran capacidad, situación característica de este proyecto, donde el tamaño del conjunto de datos era el principal cuello de botella. Para mitigarlo se emplean técnicas como la regularización, el abandono (dropout) de neuronas durante el entrenamiento, la parada temprana (early stopping) o la validación cruzada, varias de las cuales se aplican en este trabajo.

3.4 Redes neuronales recurrentes (RNN, LSTM y BiLSTM)

Una red neuronal recurrente (RNN) es un tipo de red neuronal diseñado para procesar datos secuenciales, es decir, datos en los que el orden importa y cada elemento se relaciona con los anteriores. A diferencia de las redes tradicionales, que tratan cada entrada de forma independiente, una RNN procesa la secuencia elemento a elemento manteniendo un estado interno, es decir, como una especie de "memoria" que resume la

información de los pasos previos. En cada paso, la salida depende tanto de la entrada actual como de ese estado acumulado, lo que le permite capturar dependencias temporales.

Esta característica la hace especialmente adecuada para las series temporales: secuencias de datos ordenados en el tiempo. En este proyecto, la entrada es precisamente una serie temporal, esta entrada es la evolución de las señales biomecánicas fotograma a fotograma a lo largo de la sentadilla, por lo que el análisis de cada instante se beneficia de conocer lo ocurrido en los instantes anteriores.

3.4.1 LSTM (Long Short-Term Memory)

Las RNN básicas presentan una limitación: tienen dificultades para retener información durante secuencias largas por lo que tienden a "olvidar" las dependencias lejanas. La LSTM [Hochreiter y Schmidhuber, 1997] es un tipo de RNN diseñada para resolver este problema. Incorpora un mecanismo de puertas que regulan, de forma aprendida, qué información se conserva, cuál se olvida y cuál se incorpora en cada paso. Gracias a ello, la LSTM mantiene la información relevante a lo largo de secuencias más largas, capturando dependencias temporales tanto a corto como a largo plazo.

3.4.2 BiLSTM (LSTM bidireccional)

Una RNN o una LSTM convencional procesa la secuencia en un único sentido, del principio al final, de modo que la salida de cada paso solo dispone del contexto pasado. La BiLSTM [Schuster y Paliwal, 1997] procesa la secuencia en los dos sentidos a la vez: una LSTM la recorre hacia delante (del primer fotograma al último) y otra hacia atrás (del último al primero), y sus resultados se combinan. De este modo, la decisión sobre cada elemento tiene en cuenta tanto el contexto pasado como el futuro.

Este carácter bidireccional resulta especialmente valioso para la tarea de este sistema, la segmentación de fases. Etiquetar correctamente un fotograma como "bajando" o "abajo" es más fiable si se conoce no solo lo que ha ocurrido antes, sino también lo que ocurre después, es decir si observamos un fotograma con el usuario a media sentadilla, sin el contexto previo no sabremos si esta bajando o subiendo, será determinante saber el pasado (si parte de posición de pie o de fondo) para concretar cual es la naturaleza de ese fotograma. Como el análisis se realiza a posteriori sobre el vídeo ya grabado, y no en tiempo real, la secuencia completa está disponible, por lo que el modelo puede aprovechar

el contexto futuro sin penalización. Por ello se eligió una BiLSTM para segmentar las fases de la sentadilla y delimitar las repeticiones.

3.5 Modelos de árboles y gradient boosting (XGBoost)

3.5.1 Árboles de decisión

Un árbol de decisión es un modelo de aprendizaje supervisado que realiza una predicción mediante una jerarquía de preguntas simples sobre las características de la entrada. Funciona de forma análoga a un diagrama de flujo: cada nodo interno plantea una pregunta sobre una característica frente a un umbral (por ejemplo, "¿es el ángulo de profundidad menor que un valor x ?") y, según la respuesta, sigue una rama u otra, hasta llegar a una hoja que proporciona la predicción final (la clase). Es un modelo intuitivo e interpretable, pero un árbol aislado tiene una precisión limitada y tiende al sobreajuste.

3.5.2 Conjuntos de árboles y boosting

Para superar las limitaciones de un único árbol, los métodos combinan muchos árboles. El boosting es un método el cual construye árboles de forma secuencial: cada nuevo árbol se entrena para corregir los errores cometidos por el conjunto de árboles anteriores, y la predicción final es la combinación de todos ellos. De este modo, muchos árboles simples y poco precisos se agregan para formar un modelo conjunto robusto y preciso.

3.5.3 Gradient boosting y XGBoost

El gradient boosting es una forma concreta de boosting en la que cada nuevo árbol se ajusta para reducir el error residual del conjunto, siguiendo el gradiente de una función de pérdida. XGBoost (Extreme Gradient Boosting) [Chen y Guestrin, 2016] es una implementación eficiente y regularizada del gradient boosting, muy extendida y especialmente potente sobre datos tabulares, es decir, un conjunto de datos los cuales están estructurados en filas y columnas. En este sistema, XGBoost clasifica cada KPI por repetición a partir del vector de características de la repetición. Dos propiedades de este tipo de modelo resultaron determinantes para su elección:

Menor necesidad de datos. Frente a las redes neuronales profundas, los conjuntos de árboles son eficaces con conjuntos de datos más pequeños y de naturaleza tabular. Esto fue decisivo, ya que el tamaño del conjunto de datos era el principal cuello de botella del proyecto.

Invariantes frente al cambio de escala. Los árboles dividen cada característica mediante umbrales de forma independiente, por lo que sus divisiones dependen únicamente del orden relativo de los valores, no de su magnitud. En consecuencia, cualquier transformación de los datos deja invariantes los cortes que el árbol puede aprender, de modo que no es necesario normalizar la entrada, a diferencia de lo que ocurre con las redes neuronales.

A ello se suman otras características útiles como que trabajan de forma natural con variables de magnitudes y naturalezas distintas a la vez, manejan los valores ausentes (NaN), y proporcionan una medida de la importancia de cada característica, lo que aporta interpretabilidad al modelo.

4 Técnicas y herramientas

Este capítulo describe las técnicas y herramientas empleadas en el desarrollo de MoveInsight, así como la justificación de su elección frente a las alternativas evaluadas durante la fase de investigación previa. Se organiza siguiendo las grandes capas del sistema: el reconocimiento del movimiento, los modelos de aprendizaje automático, el backend, la aplicación móvil y la infraestructura de despliegue, además de las herramientas de desarrollo y gestión del proyecto.

4.1 Metodología de desarrollo

El proyecto se ha desarrollado siguiendo un enfoque iterativo e incremental, organizado en sprints de periodicidad aproximadamente semanal. Cada sprint perseguía un objetivo concreto: la puesta en marcha de la aplicación móvil, la investigación de los estimadores de pose, los primeros entrenamientos de la red o la construcción del conjunto de datos, entre otros; y se documentó la planificación que ha permitido mantener la trazabilidad de las decisiones a lo largo del desarrollo.

La planificación temporal, la estimación de tamaño y esfuerzo (puntos de historia y UCP) y el diagrama de Gantt se recogen en el Anexo III (Estimación del tamaño y esfuerzo). Los roles de Scrum y las técnicas de obtención de requisitos se describen en el Anexo I (Especificaciones del sistema).

El carácter investigativo del proyecto, especialmente en su componente de inteligencia artificial, hacía inviable una planificación cerrada desde el inicio, ya que muchas decisiones surgieron de los resultados de iteraciones anteriores. Este enfoque iterativo permitió reorientar el trabajo cuando fue necesario, como ocurrió al pasar de un único modelo a la arquitectura final de dos etapas (BiLSTM y XGBoost).

El desarrollo se apoyó, además, en el seguimiento continuo con la tutora del trabajo y en el asesoramiento de personas expertas externas. Por un lado, un graduado en Ciencias de la Actividad Física y del Deporte (CAFYD) aportó conocimiento biomecánico que motivó la incorporación del centro de sustentación como variable de análisis, junto con otros consejos. Por otro, una visita al centro de inteligencia artificial BISITE permitió contrastar el trabajo con investigadores del área, de cuya recomendación surgió la incorporación del modelo XGBoost.

Todo el código se ha mantenido bajo control de versiones con Git, alojado en GitLab y disponible en https://gitlab.com/diegogt_24/moveinsight.git .

4.2 Aproximaciones al reconocimiento del movimiento

Una de las primeras decisiones del proyecto fue la estrategia general para extraer información del cuerpo humano a partir del vídeo. Durante la investigación previa se revisaron diversos trabajos académicos sobre reconocimiento de imágenes y análisis de movimiento, lo que permitió descartar las aproximaciones menos adecuadas y orientar el diseño hacia la solución finalmente adoptada.

Los enfoques basados en ingeniería de características manual, como la extracción de momentos geométricos de Hu, los descriptores de tipo «bag of words» o la detección clásica por regiones y patrones de color, resultan eficaces para objetos rígidos o siluetas limpias en entornos controlados, pero son inviables para un cuerpo humano articulado y deformable en movimiento; los trabajos de Pardo Feijoo [Pardo Feijoo], García [García] y Santa María et al. [Santa María et al.] ilustran estas limitaciones. Por el contrario, los trabajos basados en aprendizaje profundo y redes neuronales convolucionales, como los de Oliva Rodríguez [Oliva Rodríguez] y Martínez Llamas [Martínez Llamas], confirman que estas constituyen la vía fiable para entornos complejos, si bien advierten de su elevado coste computacional y de que la escasez de datos es el principal cuello de botella, mitigable mediante técnicas de transferencia de aprendizaje y aumento de datos.

De este análisis se extrajo la estrategia adoptada: en lugar de entrenar un detector genérico desde cero, se optó por emplear un modelo especializado de estimación de pose, ya entrenado, que devuelve directamente los puntos clave del cuerpo, y construir el análisis biomecánico sobre dichos puntos. Esta decisión evita reinventar la detección del cuerpo humano y concentra el esfuerzo en la parte específica del problema: la evaluación de la técnica de la sentadilla.

4.3 Visión por ordenador: OpenCV y MediaPipe BlazePose

Para el tratamiento del vídeo se emplea OpenCV [Bradski, 2000], ya que es una biblioteca de referencia en visión por ordenador en Python por su madurez, rendimiento y amplia documentación, como se ha podido descubrir en la investigación previa. En este proyecto se utiliza para la lectura de los fotogramas, los cambios de espacio de color y el posterior renderizado del esqueleto sobre el vídeo, así como para obtener los fps del vídeo,

los fps se usarán en distintos cálculos a lo largo del proceso, más concretamente a la hora de calcular velocidades.

Para la estimación de la pose se eligió BlazePose [Bazarevsky et al., 2020], integrado en el framework MediaPipe [Lugaresi et al., 2019]. La razón determinante de esta elección es que BlazePose proporciona coordenadas tridimensionales (X, Y, Z) de los puntos clave del cuerpo, frente al carácter bidimensional de la mayoría de las alternativas. Esta tercera dimensión resulta imprescindible para calcular con precisión ángulos articulares como la flexión de la rodilla o la inclinación del torso en un movimiento como la sentadilla, en el cual el plano 2D aporta poca información a la hora de su realización.

4.3.1 Alternativas evaluadas

Antes de optar por BlazePose se consideraron otras soluciones de estimación de pose:

- OpenPose y AlphaPose: soluciones precisas, pero computacionalmente pesadas y orientadas a la salida bidimensional, lo que las hacía poco adecuadas para los objetivos del proyecto.
- MoveNet: muy eficiente, pero limitado a coordenadas en dos dimensiones, insuficientes para el cálculo fiable de ángulos tridimensionales.
- YOLO-Pose y YOLO-NAS Pose: excelente equilibrio entre velocidad y precisión; constituyen una alternativa sólida, por lo que se consideró como un posible plan B en caso de orientar el proyecto hacia un entorno más industrial.
- RTMPose y ViTPose: modelos del estado del arte, muy precisos, pero con unos requisitos de cómputo y una complejidad de configuración excesivos para el alcance de este trabajo.

BlazePose ofrecía el mejor compromiso entre precisión, salida tridimensional y viabilidad, por lo que se adoptó como tecnología central de la fase de visión.

4.4 Modelos de aprendizaje automático

El análisis de la técnica se apoya en modelos de aprendizaje automático que operan sobre las series temporales de las señales biomecánicas. Durante la investigación se evaluaron distintas arquitecturas, desde las más ligeras (GRU, 1D CNN, LSTM), idóneas para ejecutarse en el propio dispositivo, hasta las más pesadas (Transformer [Vaswani et al., 2017] y ST-GCN [Yan et al., 2018]), de gran capacidad pero con unos requisitos de datos y cómputo desproporcionados para la escala del proyecto. Se evaluaron en función

de complejidad, a la cantidad de datos que requieren y a su idoneidad para el móvil o el servidor. El estudio comparativo se detalla en la Tabla 3 en el capítulo Aspectos Relevantes del Desarrollo.

De ese estudio se optó por una arquitectura BiLSTM para la segmentación de las fases de la sentadilla, por su capacidad de analizar el contexto pasado y futuro de cada fotograma sin incurrir en los elevados requisitos de los modelos más complejos. Posteriormente, y por recomendación de un investigador del centro de inteligencia artificial BISITE, se incorporó un modelo XGBoost para la evaluación de los KPIs por repetición, dado que los modelos de gradiente sobre árboles requieren una cantidad de datos sensiblemente menor, el principal cuello de botella del proyecto. El sistema final combina ambos: BiLSTM para las fases y XGBoost para los KPIs.

En cuanto a las bibliotecas, la red BiLSTM se ha entrenado y ejecutado con TensorFlow y Keras, mientras que para el modelo XGBoost se ha empleado la propia biblioteca junto con scikit-learn para la validación cruzada y el cálculo de las métricas de evaluación.

El pipeline de procesamiento y los modelos se han implementado en Python, lenguaje de referencia en ciencia de datos e inteligencia artificial. Sobre él se apoyan las bibliotecas numéricas y científicas empleadas: NumPy para el cálculo vectorial, pandas para la manipulación de los datos tabulares, SciPy; en particular su implementación del filtro de Savitzky-Golay [Savitzky y Golay, 1964] para el suavizado de las señales, y Matplotlib para la generación de gráficas e informes.

4.5 Fuentes de datos para el entrenamiento

El conjunto de datos empleado para entrenar los modelos se construyó combinando vídeos procedentes de repositorios y plataformas con licencia adecuada como: Pexels, iStock, Zenodo, Kaggle y MMFit, además, de grabaciones propias realizadas específicamente para el proyecto. Estas últimas resultaron imprescindibles ante la escasez de ejemplos de ejecuciones técnicamente incorrectas, necesarios para que los modelos aprendieran a distinguir una repetición defectuosa. El proceso completo de construcción y etiquetado del conjunto de datos se detalla en el capítulo de Aspectos Relevantes del Desarrollo.

4.6 Métricas de evaluación

La evaluación de los modelos se ha realizado con las métricas habituales en problemas de clasificación: la exactitud (accuracy) global y, dado el desbalanceo entre clases, la exhaustividad (recall) y la medida F1 por clase. Permitiendo hacer una estimación de la precisión del modelo.

Un aspecto relevante en cuanto a la metodología es la forma de validar el modelo XGBoost. En lugar de una validación cruzada estándar, se ha empleado una validación cruzada agrupada por vídeo: todas las repeticiones procedentes de un mismo vídeo se mantienen juntas en el mismo pliegue, evitando que aparezcan a la vez en entrenamiento y validación. De no hacerlo así, el modelo podría reconocer en validación repeticiones muy similares a las ya vistas en entrenamiento, produciendo una fuga de información y una estimación de rendimiento artificialmente optimista. Los resultados concretos de esta evaluación se presentan en el capítulo de Aspectos Relevantes del Desarrollo.

4.7 Backend y API REST

El backend se ha construido con FastAPI, un framework web de Python moderno y de alto rendimiento, elegido por su rapidez, su tipado mediante anotaciones y la generación automática de documentación de la API. La aplicación se sirve mediante Uvicorn, un servidor ASGI. La persistencia se gestiona con el ORM SQLAlchemy sobre una base de datos MariaDB, y la validación de los datos de entrada y salida con Pydantic. La autenticación se implementa mediante JSON Web Tokens (JWT).

4.8 Aplicación móvil

La aplicación cliente se ha desarrollado de forma nativa para Android en lenguaje Kotlin, empleando Jetpack Compose como toolkit declarativo de interfaz y Material 3 como sistema de diseño. La inyección de dependencias se gestiona con Hilt, y la comunicación con el backend se realiza con Retrofit sobre OkHttp, utilizando Gson para la conversión entre JSON y objetos. La arquitectura interna de la aplicación se describe en el capítulo de Aspectos Relevantes del Desarrollo.

4.9 Infraestructura y despliegue

El sistema se despliega con la técnica de uso de contenedores mediante la plataforma Docker y docker-compose, lo que garantiza un entorno de ejecución reproducible e

independiente de la máquina anfitriona. Así evitamos una gran cantidad de posibles inconvenientes a la hora del despliegue. Se emplea Nginx como proxy inverso y para la terminación TLS, con certificados gratuitos de Let's Encrypt, y el conjunto se aloja en un servidor de la Universidad de Salamanca.

4.10 Herramientas de desarrollo y gestión del proyecto

Para el control de versiones se ha utilizado Git, con el repositorio alojado en GitLab, registrando de forma continua la evolución del código. El desarrollo de la aplicación Android se ha realizado con Android Studio, mientras que el código en Python del backend se ha desarrollado en el IDE Visual Studio Code. Por último, ante la ausencia de una herramienta adecuada para el etiquetado de los vídeos, se desarrolló una herramienta propia en Python apoyada en OpenCV, descrita en el capítulo de Aspectos Relevantes del Desarrollo, que permitió etiquetar de forma eficiente las fases y los KPIs de cada repetición.

5 Aspectos Relevantes del Desarrollo

5.1 Visión general de la arquitectura del sistema

El sistema se divide en dos grandes bloques: el backend y el frontend. El backend se encarga del procesamiento de los vídeos, de la extracción de los puntos del cuerpo humano y de la inferencia de los modelos de inteligencia artificial entrenados. Mientras que el frontend, por su parte, es una aplicación Android que constituye el punto de entrada del usuario: desde ella se graban o suben los vídeos de la sentadilla y, posteriormente, se consultan el análisis, las recomendaciones, los informes y las gráficas.

El flujo general es el siguiente: el usuario graba un vídeo de su sentadilla desde la aplicación móvil y este se envía al servidor, alojado en la infraestructura de la Universidad. Allí se procesa para extraer los puntos clave del cuerpo y se calcula una serie de variables biomecánicas que constituyen la entrada de los modelos encargados de evaluar la ejecución. Finalmente, los resultados se almacenan en la base de datos y se devuelven a la aplicación.

El análisis y el diseño completos (modelo de dominio, diagramas de secuencia, modelo C4 y decisiones arquitectónicas) se desarrollan en el Anexo II (Análisis y diseño del sistema).

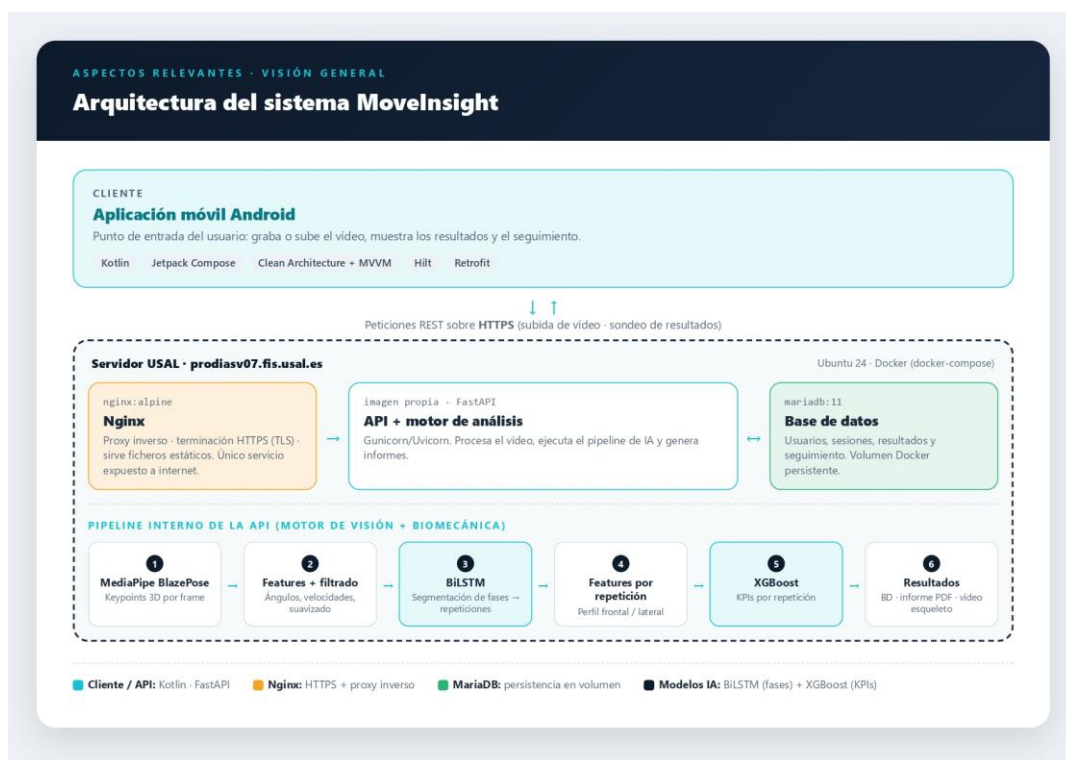


Figura 2. Arquitectura del sistema

5.2 Pipeline de visión artificial

El pipeline de visión artificial constituye el primer eslabón del backend y, posiblemente, el más determinante de todo el sistema, ya que la calidad del análisis posterior depende directamente de la precisión con la que se extraiga la información esquelética del usuario. En este apartado se detallan las distintas alternativas exploradas durante el diseño de esta fase, así como las decisiones tomadas y su justificación. La investigación previa resultó clave en este proceso, pues permitió no solo seleccionar las tecnologías más adecuadas para los objetivos del proyecto, sino también anticipar los principales problemas a los que sería necesario hacer frente.

5.2.1 Preprocesado del vídeo

Para el preprocesado de los fotogramas se ha empleado OpenCV, la biblioteca de referencia para el tratamiento de vídeo en Python ya presentada en el capítulo anterior. En este proyecto se utiliza para la lectura de los fotogramas, los cambios de espacio de color y el posterior renderizado del esqueleto sobre el vídeo. Tal como se detalla en el apartado 3.1, no se aplica un redimensionado de los fotogramas, ya que MediaPipe realiza internamente su propio escalado al procesarlos.

1. Lectura fotograma a fotograma y cadencia — app/services/video.py

```
cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS) or 30.0      # cadencia real del vídeo (fps)
while True:
    ret, frame = cap.read()                  # lectura secuencial, fotograma a fotograma
    if not ret:
        break
    _, pose_vector = pose_processor.process_frame(frame)
cap.release()
```

2. Cambio de espacio de color (BGR → RGB) — processing/pose_processor.py

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # OpenCV usa BGR; MediaPipe espera RGB
mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)
```

3. Renderizado del esqueleto sobre el vídeo — processing/visualization.py

```
cv2.line(out, puntos[a], puntos[b], color, grosor_linea, cv2.LINE_AA) # huesos
cv2.circle(out, (px, py), radio_punto, (255, 255, 255), -1, cv2.LINE_AA) # articulaciones
```

5.2.2 Estimación de pose, elección de BlazePose de Mediapipe

El estudio comparativo de los distintos estimadores de pose y la justificación de la elección se desarrollan en el capítulo de Técnicas y herramientas; en este apartado se recupera únicamente la decisión adoptada y su fundamento en el contexto del pipeline.

Finalmente se optó por BlazePose, el modelo específico que MediaPipe emplea para detectar el cuerpo y extraer los 33 puntos clave. Su ventaja determinante frente al resto de alternativas es la salida tridimensional (X, Y, Z). En el contexto del pipeline, lo relevante es su consecuencia práctica: BlazePose permite obtener un resultado aproximado al que se conseguiría disponiendo de dos cámaras simultáneas, una en el plano frontal y otra en el lateral, si bien con las limitaciones propias de inferir la profundidad a partir de una única vista.

Otra decisión clave ha sido que modelo de BlazePose usar, ya que MediaPipe ofrece 3 modelos, cada modelo tiene pros y contras. Si el modelo tiene mayor precisión, el tiempo de inferencia de los resultados será mayor. En nuestro caso la decisión ha sido priorizar precisión frente a rapidez por lo que el modelo elegido ha sido el modelo “heavy” el más preciso, pero también el más lento. De esta forma nos aseguramos de ofrecer al usuario un resultado óptimo, aunque esto implique un mayor tiempo de espera.

```
# backend/processing/pose_processor.py
class PoseProcessor:
    def __init__(self, fps, model_path='modelos/pose_landmarker_heavy.task'):
        base_options = python.BaseOptions(model_asset_path=model_path) # modelo "heavy"
        options = vision.PoseLandmarkerOptions(
            base_options=base_options,
            running_mode=vision.RunningMode.VIDEO, # analisis de video
            min_pose_detection_confidence=0.5, min_tracking_confidence=0.5)
        self.detector = vision.PoseLandmarker.create_from_options(options)
```

5.2.3 Tratamiento de la fiabilidad

La tecnología de MediaPipe ofrece resultados notables, pero conviene ser realista respecto a sus límites. La estimación de las coordenadas Z a partir de una única cámara en un solo plano es, por su propia naturaleza, una tarea compleja en la que resulta difícil alcanzar una precisión elevada de forma consistente. En determinados fotogramas, especialmente cuando una parte del cuerpo queda ocluida o fuera del plano óptimo, MediaPipe genera coordenadas con escasa fiabilidad.

Para mitigar este problema se ha implementado un filtro basado en el propio indicador de confianza del modelo. MediaPipe devuelve, por cada fotograma y cada punto clave, un vector que contiene las coordenadas (X, Y, Z) junto con un valor de fiabilidad asociado. Estableciendo un umbral sobre dicho valor, el sistema descarta aquellas detecciones poco fiables y conserva únicamente las que superan el umbral establecido. Cuando un punto no alcanza la confianza mínima, su valor no se sustituye por una estimación dudosa, sino que se marca como dato ausente para su tratamiento posterior. Este mecanismo permite explotar la capacidad de trabajar con coordenadas tridimensionales sin comprometer en exceso la fiabilidad global del análisis, ofreciendo en muchos casos una valoración cercana a un nivel profesional y descartando aquellos datos que no resultan realistas.

Este filtro de fiabilidad se materializa en la función `calcular_articulacion_segura()`, que solo calcula un ángulo si los tres puntos implicados superan el umbral de visibilidad; en caso contrario devuelve un valor ausente (NaN).

```
# backend/processing/pose_features.py
def calcular_articulacion_segura(kp, indices, umbral=UMBRAL_CONFIANZA):
    for idx in indices:
        if kp[idx][3] < umbral:      # kp[idx][3] = fiabilidad del punto
            return np.nan           # punto poco fiable -> dato ausente
    return calculate_angle_3d(kp[indices[0]]
                              ,kp[indices[1]],kp[indices[2]])
```

5.3 Motor de biomecánica

El motor de biomecánica constituye el núcleo analítico de la aplicación: es el responsable de transformar los datos de pose en bruto que produce MediaPipe en una evaluación técnica y cuantitativa de cada repetición de la sentadilla. A continuación, se describe el flujo general de tratamiento de los datos.

Los datos obtenidos por MediaPipe no se utilizan directamente, ya que la detección de pose introduce ruido y, ocasionalmente, fotogramas sin detección. Por ello, antes de cualquier cálculo se aplica una etapa de filtrado y suavizado de las señales, que se detalla en el subapartado correspondiente.

A partir de las coordenadas ya tratadas se calcula un primer conjunto de parámetros biomecánicos por fotograma. De este conjunto, un subgrupo de señales se normaliza y constituye la entrada de la red BiLSTM. La red BiLSTM recibe esta secuencia temporal y realiza una clasificación fotograma a fotograma, asignando a cada instante una de las cuatro fases de la sentadilla: de pie, descenso, abajo y ascenso. A partir de esta secuencia

de fases se delimitan las repeticiones individuales, identificando cada repetición válida como un ciclo completo de bajando → abajo → ascenso.

Una vez segmentada cada repetición, se calcula sobre ella un segundo conjunto de parámetros, esta vez agregados a nivel de repetición y no de fotograma. Este vector de características por repetición se introduce finalmente en el modelo XGBoost, encargado de emitir la valoración de los distintos indicadores de rendimiento (KPIs).

Conviene precisar que el sistema dispone de dos variantes de este modelo, una para el plano frontal y otra para el lateral, y que la elección entre ellas depende del plano desde el que se ha grabado el vídeo.

5.3.1 *Filtros, suavizado y normalización*

Las coordenadas de pose en bruto contienen ruido, fotogramas sin detección o nulos y dependencias del encuadre que degradarían tanto la segmentación de fases como la evaluación de la técnica. Por ello, antes y durante el cálculo de las características se aplican varias etapas de filtrado, suavizado y normalización, que se describen a continuación.

Filtros

El primer filtro ya ha sido mencionado en el apartado anterior y se trata del filtro de fiabilidad, en el que todos los puntos deben superar un umbral de confianza determinado, si no ese punto se declara como no disponible, este filtro se aplica a la hora de calcular distintas métricas biomecánicas.

El segundo filtro es el que soluciona estos huecos de métricas no disponibles, para ello usa la interpolación lineal, es decir, se rellenan los huecos (rellena interpolando entre el último valor fiable anterior y el primero posterior al hueco) de hasta 10 fotogramas (límite investigado para que la fiabilidad no se degrade y la red no se invente valores). El motivo es recuperar la continuidad temporal de la señal ante oclusiones breves habituales cuando un brazo cruza por delante de la cadera, por ejemplo, sin inventar datos en ausencias prolongadas: los vacíos más largos se conservan como no disponibles y se tratan como valores ausentes.

```
# backend/processing/pose_features.py
UMBRAL_CONFianza = 0.65                                # filtro de fiabilidad (descarta puntos dudosos)
# Filtro de huecos: interpola linealmente los tramos de hasta 10 fotogramas
df = df.interpolate(method='linear', limit=10, limit_direction='both')
```

Suavizado

Para suavizar la señal usamos el filtro de Savitzky-Golay, sobre la señal ya interpolada se aplica este filtro, el cual, ajusta un polinomio local a cada tramo, lo que atenúa el ruido de alta frecuencia preservando la forma del movimiento, es decir, sin aplanar los picos reales del movimiento, a diferencia de una media móvil. Su aplicación es de vital importancia ya que, por un lado reduce el temblor habitual de la estimación de pose y por otro lado es imprescindible realizarlo antes de derivar, ya que las velocidades y aceleraciones se calculan por diferenciación numérica, sin el filtro estas magnitudes serían inservibles.



Figura 3. Proceso de suavizado

En el código, el suavizado se aplica con el filtro de Savitzky-Golay sobre cada señal, y la normalización z-score de la entrada de la BiLSTM emplea los estadísticos del conjunto de entrenamiento:

```
# backend/processing/pose_features.py
df[col] = savgol_filter(temp_col, window_length=11, polyorder=2)
```

Otro suavizado utilizado es el suavizado de la secuencia de fases en el cual se aplica el voto por mayoría. La BiLSTM produce una etiqueta de fase por fotograma, y esa salida puede contener cambios espurios de muy corta duración (por ejemplo, un único fotograma

clasificado erróneamente como abajo en mitad del descenso). Para corregirlo se aplica un suavizado por voto mayoritario en una ventana deslizante de 7 fotogramas, que asigna a cada instante la fase más frecuente en su entorno y elimina las transiciones más breves que la mitad de la ventana. Se utiliza un voto por mayoría, y no un filtro numérico como el de Savitzky-Golay, porque se opera sobre etiquetas categóricas (las cuatro fases) y no sobre señales continuas. Su motivo es robustecer la posterior delimitación de repeticiones, que depende de detectar transiciones de fase limpias.

```
# backend/processing/phase_predictor.py
def _smooth_phases(phases, window=7):          # voto por mayoria en ventana deslizante
    half = window // 2
    out = phases.copy()
    for i in range(half, len(phases) - half):
        ventana = phases[i - half : i + half + 1]
        out[i] = int(np.argmax(np.bincount(ventana, minlength=4)))
    return out
```

Normalización

La primera normalización aplicada es la normalización geométrica. Varias características se expresan en magnitudes relativas y no absolutas, durante su propio cálculo. El desplazamiento del centro de sustentación se divide por la longitud del torso, y la desviación de valgo se expresa como fracción de la longitud de la pierna. El motivo es hacer estas métricas invariantes a la estatura del usuario y a su distancia a la cámara: dos personas de distinta altura, o una misma persona grabada más cerca o más lejos, producirían valores absolutos diferentes para una técnica idéntica. Al normalizar por una referencia anatómica del propio sujeto, la característica describe el gesto y no las condiciones de grabación.

```
# backend/processing/pose_features.py (extract_features)
# (1) centro de sustentacion relativo a la longitud del torso
longitud_torso = np.linalg.norm(mid_shoulder - mid_hip)
delta = (sust - mid_hip) / longitud_torso
# (2) valgo expresado como fraccion de la longitud de la pierna
return float(ml_deviation / line_len)
```

Otra normalización aplicada es la normalización estadística z-score. Inmediatamente antes de introducir la secuencia en la red BiLSTM se aplica una normalización z-score por característica, restando la media y dividiendo por la desviación típica:

```
(seq_norm = (seq - mu) / std)
```


Un aspecto fundamental es que μ y σ no se calculan sobre el vídeo de entrada, sino que son los estadísticos del conjunto de entrenamiento, almacenados junto al modelo y cargados una sola vez. De este modo se aplica exactamente la misma transformación que durante el entrenamiento, evitando el desajuste entre entrenamiento e inferencia. El motivo es que las redes neuronales aprenden mejor cuando todas sus entradas están en una escala parecida. Si una característica toma valores muy grandes (por ejemplo, ángulos de 0 a 180) y otra valores muy pequeños (por ejemplo, desplazamientos de 0 a 1), la red tiende a fijarse demasiado en la de mayor magnitud y a ignorar la pequeña, aunque ambas sean igual de importantes. Al normalizar, todas las características pasan a moverse en el mismo rango, de modo que la red las trata por igual y el entrenamiento es más rápido y estable.

Otro aspecto para tener en cuenta es la ausencia de normalización a la hora de introducir parámetros en el modelo XGBoost. La explicación es que XGBoost, es un modelo basado en árboles de decisión, cuyos nodos dividen el espacio mediante umbrales sobre cada característica de forma independiente (del tipo «¿es la profundidad menor que un valor x ?»). Estas divisiones dependen únicamente del orden relativo de los valores, no de su escala: cualquier transformación monótona como restar una media o dividir por una desviación típica deja invariantes los puntos de corte que el árbol puede aprender y, por tanto, sus predicciones. Normalizar no aportaría ninguna mejora y solo añadiría un paso de preprocesado innecesario.

Etapas	Técnica	Dónde	Por qué
Filtro fiabilidad	Umbral visibilidad 0,65	pose_features (por frame)	Eliminar ángulos fantasma
Filtro huecos	Interpolación lineal (≤ 10 frames)	pose_features	Recuperar continuidad ante oclusiones breves
Suavizado señales	Savitzky-Golay (ventana 11, orden 2)	pose_features, antes de derivar	Reducir ruido sin deformar el gesto; permitir derivar vel/acc
Suavizado fases	Voto mayoritario (ventana 7)	phase_predictor, tras la BiLSTM	Limpiar transiciones espurias de fase
Norm. geométrica	División por torso/pierna	pose_features	Invarianza a estatura y distancia a cámara
Norm. z-score	$(x-\mu)/\sigma$ con stats de entrenamiento	phase_predictor, entrada BiLSTM	Convergencia estable de la red
XGBoost	Sin normalización	—	Los árboles son invariantes a la escala

Tabla 2. Resumen de filtros, suavizado y normalización en el sistema

5.3.2 Cálculos de métricas

En este apartado se describen las métricas calculadas a partir de los datos esqueléticos y cómo su número y naturaleza han ido evolucionando a lo largo del desarrollo del sistema, en paralelo a los cambios introducidos en los modelos.

Como se detallará más adelante, las primeras versiones del backend incorporaban únicamente la red BiLSTM, que se encargaba simultáneamente de la segmentación de las repeticiones y de la obtención de las métricas de calidad del movimiento. Posteriormente, el sistema evolucionó hacia un esquema que incorporaba además un modelo XGBoost. Del mismo modo que los modelos fueron cambiando durante el proceso, también lo hizo el conjunto de métricas que se proporcionaba como entrada a las redes.

Inicialmente solo se pasaban los ángulos de cadera, rodilla, tobillo, hombro y codo. La inclusión de hombro y codo respondía a la intención de que el sistema pudiera contemplar un mayor número de ejercicios y no únicamente la sentadilla; más adelante se explicará por qué este objetivo no resultó finalmente viable. La primera evolución consistió precisamente en eliminar los ángulos irrelevantes para el caso de estudio hombro y codo, reduciendo así la entrada a la información estrictamente necesaria para el análisis de la sentadilla.

Una segunda mejora significativa fue la introducción de las coordenadas del centro de sustentación. Esta incorporación surgió a raíz del consejo de un graduado en Ciencias de la Actividad Física y del Deporte (CAFYD), quien puso de manifiesto la importancia del centro de sustentación en el análisis de ejercicios dinámicos como la sentadilla. La combinación de ambas decisiones la eliminación de la información redundante y la inclusión del centro de sustentación supuso una mejora sustancial en el rendimiento del sistema.

El planteamiento cambió de forma notable cuando, pese a que los resultados mejoraban, estos seguían sin alcanzar las expectativas. Este punto de inflexión motivó la incorporación del modelo XGBoost y, con el fin de facilitar su trabajo, se optó por calcular un conjunto más amplio de métricas que la red pudiera interpretar. En la figura siguiente se recogen todas las métricas calculadas y empleadas en ambas redes.

MOTOR DE BIOMECÁNICA · CARACTERÍSTICAS DE ENTRADA

Características que recibe cada modelo

El sistema emplea dos modelos con entradas de naturaleza distinta: la **BiLSTM** recibe una **serie temporal por fotograma** (8 señales) para segmentar las fases de la sentadilla, mientras que **XGBoost** recibe un **vector agregado por repetición** (37 estadísticos) para evaluar la calidad técnica.

Red BiLSTM — Segmentación de fases 8 features · por fotograma

Secuencia temporal normalizada (z-score). Cada señal aporta **un valor por cada fotograma** del vídeo.

L_knee Ángulo rodilla izquierda	R_knee Ángulo rodilla derecha	L_hip Ángulo cadera izquierda	R_hip Ángulo cadera derecha
sust_dy Desplazamiento vertical del centro de sustentación	vel_knee Velocidad angular de rodilla	vel_y_hip Velocidad vertical de cadera	acc_y_hip Aceleración vertical de cadera

Red XGBoost — Evaluación de KPIs 37 features · por repetición

Estadísticos calculados sobre cada repetición ya segmentada. **Sin normalización** (los árboles son invariantes a la escala).

Profundidad 3D 4 min_L_knee Ángulo mínimo de rodilla izquierda (máxima flexión, 3D) min_R_knee Ángulo mínimo de rodilla derecha (3D) min_hip_angle Ángulo mínimo de cadera (flexión máxima) knee_depth_avg Media de la flexión mínima de ambas rodillas	Profundidad 2D 3 min_L_knee_2d Flexión mínima de rodilla izq. en el plano XY (robusta en lateral) min_R_knee_2d Flexión mínima de rodilla der. en el plano XY knee_depth_avg_2d Media de la flexión mínima 2D de ambas rodillas	Torso 4 max_torso Inclinación máxima del torso respecto a la vertical p90_torso Percentil 90 de la inclinación del torso (máximo robusto) mean_torso_bottom Inclinación media del torso en la fase de fondo torso_range Recorrido de la inclinación del torso (máx - mín)
Rodillas (valgo proxy) 4 knee_sym_at_bottom Diferencia de ángulo entre rodillas en la fase de fondo knee_diff_max Diferencia máxima de ángulo entre ambas rodillas ankle_L_min Ángulo mínimo del tobillo izquierdo ankle_R_min Ángulo mínimo del tobillo derecho	Valgo real 4 valgus_L_peak Pico de desviación medial de la rodilla izquierda valgus_R_peak Pico de desviación medial de la rodilla derecha valgus_bottom_mean Media del valgo (módulo) en la fase de fondo valgus_bilateral Valgo bilateral signado (ambas rodillas hacia dentro)	Simetría 5 knee_diff_integral Diferencia entre rodillas acumulada en toda la rep. (normalizada) hip_diff_mean Diferencia media entre los ángulos de cadera izq/der. hip_diff_max Diferencia máxima entre los ángulos de cadera L_knee_range Recorrido angular de la rodilla izquierda R_knee_range Recorrido angular de la rodilla derecha
Ritmo 8 descent_speed_peak Velocidad máxima durante el descenso ascent_speed_peak Velocidad máxima durante el ascenso descent_ratio Fracción del tiempo dedicada al descenso (tempo) jerk_mean Jerk medio (suavidad global del movimiento) jerk_max Jerk máximo (sacudida más brusca) total_duration_s Duración total de la repetición (segundos) descent_frames Nº de fotogramas de la fase de descenso ascent_frames Nº de fotogramas de la fase de ascenso	Estabilidad 4 sust_dx_range Recorrido lateral del centro de sustentación sust_dx_std Oscilación lateral (desv. típica) del centro de sustentación sust_dy_range Recorrido vertical del centro de sustentación sust_dz_range Recorrido en profundidad del centro de sustentación	Pista de vista 1 hip_zx_ratio_mean Orientación media de la cámara (0 = frontal, 1 = lateral)

● **BiLSTM: 8** señales · entrada temporal · normalización z-score

● **XGBoost: 37** estadísticos · entrada por repetición · sin normalización

Figura 4. Descripción de las métricas utilizadas en ambas redes neuronales

De las 37 métricas calculadas, se describen a continuación cinco representativas junto con el fragmento de código que las implementa.

calculate_angle_3d(): Utilizado para calcular el ángulo de rodilla, el ángulo de rodilla es un valor clave tanto como para determinar las fases de la sentadilla como para determinar la profundidad de esta. Para ello se toman tres puntos (cadera, rodilla y tobillo), se forman los vectores que parten de la rodilla y se obtiene el ángulo a partir del producto escalar y el arcocoseno. La profundidad del KPI es el ángulo mínimo de rodilla en la repetición.

```
# backend/processing/pose_features.py
def calculate_angle_3d(a, b, c):
    a, b, c = np.array(a[:3]), np.array(b[:3]), np.array(c[:3])
    ba = a - b; bc = c - b
    cosine_angle = np.dot(ba, bc)/np.linalg.norm(ba)* np.linalg.norm(bc))
    return np.degrees(np.arccos(np.clip(cosine_angle, -1.0, 1.0)))
```

calculate_torso_3d(): Utilizado para calcular el ángulo del torso respecto a la vertical, se utiliza principalmente para medir la inclinación del torso, y que el modelo XGBoost decida junto con otros parámetros pero tomando este como principal si el ángulo es bueno o preocupa la excesiva inclinación. Se calcula el vector hombros-caderas y se mide su ángulo respecto a la vertical de referencia [0, -1, 0] (negativa porque en MediaPipe el eje Y crece hacia abajo).

```
# backend/processing/pose_features.py
calculate_torso_3d(shoulder_mid, hip_mid):
    v_torso = shoulder_mid - hip_mid
    v_vertical = np.array([0, -1, 0])
    cos = np.dot(v_torso, v_vertical) / (np.linalg.norm(v_torso) * np.linalg.norm(v_vertical))
    return np.degrees(np.arccos(np.clip(cos, -1.0, 1.0)))
```

calcular_valgus_lateral(): Calcula cuánto se desvía la rodilla hacia dentro o hacia fuera respecto a la línea recta que une la cadera con el tobillo. Se descompone el vector cadera-rodilla en su componente perpendicular a la línea cadera-tobillo y se proyecta sobre el eje medial-lateral del cuerpo, aislando la desviación frontal pura. Es invariante a la orientación de la cámara ya que se proyecta sobre el eje del cuerpo y no de la imagen.

```
# backend/processing/pose_features.py
def calcular_valgus_lateral(kp, hip_idx, knee_idx, ankle_idx, umbral=UMBRAL_CONFIANZA):
    # 1. Fiabilidad: los 5 puntos deben ser visibles
    required = (hip_idx, knee_idx, ankle_idx, L_HP_IDX, R_HP_IDX)
    if any(kp[i][3] < umbral for i in required):
        return np.nan
    hip, knee, ankle = kp[hip_idx][:3], kp[knee_idx][:3], kp[ankle_idx][:3]
```

```

# 2. Eje medial-lateral del cuerpo (L_hip -> R_hip), normalizado    ml_axis =
kp[R_HP_IDX][:3] - kp[L_HP_IDX][:3]
ml_axis = ml_axis / np.linalg.norm(ml_axis)
# 3. Línea "ideal" cadera -> tobillo
line      = ankle - hip
line_len = np.linalg.norm(line)
line_dir = line / line_len
# 4. Componente de la rodilla perpendicular a esa línea
knee_rel = knee - hip
perp_vec = knee_rel - np.dot(knee_rel, line_dir) * line_dir
# 5. Proyección sobre el eje ML -> aísla la desviación lateral (valgo/varo)
ml_deviation = np.dot(perp_vec, ml_axis)
return float(ml_deviation / line_len) # fracción de la longitud de la pierna

```

calcular_centro_sustentacion(): Utilizado para localizar el centro de la base de apoyo del usuario (el promedio de los puntos del pie) y medir su desplazamiento a lo largo de la sentadilla, lo que refleja el equilibrio y la estabilidad del movimiento. No se corresponde con un KPI mostrado directamente al usuario, sino que actúa como información de apoyo en dos puntos del sistema. Indica si el usuario mantiene el peso controlado o se balancea durante la repetición, muy útil además si en un futuro se quisiera introducir más KPIs, como la estabilidad. Se calcula como la media de los puntos del pie fiables (tobillos, talones y punteras), expresada de forma relativa a la cadera y normalizada por la longitud del torso, lo que la hace invariante a la estatura y a la distancia a la cámara.

```

# backend/processing/pose_features.py
def calcular_centro_sustentacion(kp, umbral=UMBRAL_CONFIANZA):
    """Calcula el promedio de los puntos de apoyo que sean fiables."""
    # Puntos de la base: Tobillos, Talones y Punteras
    foot_indices = [27, 28, 29, 30, 31, 32]
    puntos_validos = [kp[i][:3] for i in foot_indices if kp[i][3] >= umbral]

    if len(puntos_validos) < 2: # Necesitamos al menos 2 puntos para una base mínima
        return np.nan, np.nan, np.nan

    return np.mean(puntos_validos, axis=0)

```

extract_rep_features(): Es una función la cual engloba el conjunto de cálculos por repetición, es decir, los cálculos de esta función son realizados una vez el ejercicio ya ha sido segmentado en repeticiones. Como ejemplo vamos a visualizar como se calcula el ritmo de cada repetición:

Se localiza el fotograma de máxima profundidad (mínimo del ángulo de rodilla), que divide la repetición en bajada y subida, y sobre la velocidad vertical se toman los picos de cada fase.

```
# backend/processing/rep_features.py
valley_idx = int(np.nanargmin(knee_min_sig))          # fotograma mas profundo
descent_ratio = valley_idx / n                       # proporcion de bajada
descent_speed_peak = float(np.nanmax(np.abs(vel[:valley_idx]))) # pico bajando
ascent_speed_peak = float(np.nanmax(np.abs(vel[valley_idx:]))) # pico subiendo
```

5.3.3 Detección del perfil

El sistema realiza una detección automática del perfil de grabación con el fin de distinguir entre una vista frontal y una vista lateral, y seleccionar en consecuencia el tipo de análisis que se aplicará. Esta distinción es necesaria porque no todos los indicadores son medibles desde cualquier ángulo: en el perfil lateral se excluyen del análisis el valgo de rodilla y la simetría. Aunque inicialmente se intentó mantener estos dos parámetros también en la vista lateral, se comprobó que resultaba prácticamente inviable obtener una medida fiable. Pese a que MediaPipe proporciona coordenadas tridimensionales, cuando uno de los lados del cuerpo queda ocluido la estimación de los puntos correspondientes deja de ser fiable, y forzar un valor en esas condiciones daría lugar a una valoración deshonestas. Por ello se optó por excluir explícitamente estos indicadores en el plano lateral, en lugar de ofrecer un resultado poco riguroso.

El diseño busca, además, que el perfil frontal sea el predominante, ya que es el único que permite un análisis completo. Por este motivo, en los perfiles oblicuos se aplica un criterio favorable a la vista frontal: si en la grabación son visibles con claridad ambas rodillas y ambos hombros, el perfil se clasifica como frontal, puesto que en ese caso sí es posible captar correctamente todos los puntos necesarios para evaluar la técnica al completo. El ajuste de este comportamiento requirió un proceso de prueba y error hasta determinar los umbrales que producían una clasificación correcta para los casos de uso reales de la aplicación.

La detección se apoya en una idea geométrica sencilla. La línea que une ambas caderas (y, análogamente, la que une ambos hombros) se proyecta de forma distinta según la orientación de la cámara: En una vista frontal, esa línea recorre la imagen horizontalmente, por lo que los dos puntos presentan una gran separación en el eje X de la imagen y muy pequeña en profundidad (eje Z). En una vista lateral, el cuerpo está de costado: las dos caderas se proyectan casi en la misma posición horizontal (X similar) pero separadas en profundidad (gran diferencia en Z). A partir de esta observación, el

algoritmo recorre todos los fotogramas en los que hombros y caderas son detectables y mide, para cada uno, la separación media de hombros y caderas en el eje X y en el eje Z. Para robustecer la medida frente a fotogramas ruidosos, se toma la mediana de estos valores a lo largo de todo el vídeo y se calcula el cociente:

```
ratio = separación X / separación Z
```

Un ratio alto indica una vista frontal y un ratio bajo, una vista lateral, añadiendo al ratio la visibilidad de rodillas y hombros podemos conseguir con gran precisión el resultado deseado.

La selección del plano se implementa en la función detect_view(), que combina el cociente geométrico con la visibilidad de rodillas y hombros mediante una cascada de decisión:

```
# backend/processing/pose_features.py
ratio = avg_x / (avg_z + 1e-6)      # separacion caderas en X / en Z
if ratio > 1.6:                     # geometria claramente frontal
    return "frontal", ratio
if bilateral_fuerte and ratio >= 0.35: # ambos lados muy visibles -> frontal
    return "frontal", ratio
if ratio < 0.35:                   # un lado oculto -> lateral
    return "lateral", ratio
# zona oblicua: se decide por visibilidad de rodillas y hombros
return ("frontal", ratio) if (ambas_rodillas and ambos_hombros) else ("lateral", ratio)
```



Figura 5. Detección del perfil de grabación

5.4 Construcción del dataset

Actualmente se dispone de un conjunto de datos propio de más de 150 vídeos de sentadillas. Reunirlo ha sido posible gracias a un esfuerzo considerable, a muchas horas de búsqueda y a la colaboración de varias personas.

La construcción de este dataset ha sido, con diferencia, la parte más difícil del proyecto, y ello pese a partir sobre aviso: la investigación previa ya había puesto de manifiesto que una gran cantidad de autores se encontraba con el mismo obstáculo. Aun así, el problema resultó mayor de lo previsto.

5.4.1 Primera exploración

Al inicio del desarrollo se buscaron grandes conjuntos de datos de sentadillas, así como repositorios y plataformas de los que extraer vídeos con licencia adecuada y libres de derechos de autor. En un primer momento todo parecía sencillo, pues se logró reunir una cantidad notable de vídeos con ejecuciones razonablemente correctas. Las principales fuentes fueron Pexels, iStock, Zenodo y diversos conjuntos de datos de Kaggle y MMFit. Como primera aproximación a un dataset propio, el resultado fue aceptable.

5.4.2 El cuello de botella

Pronto, sin embargo, se advirtió cuál era el verdadero problema. De todos los vídeos recopilados hasta entonces (alrededor de 70), cerca del 90 % correspondían a sentadillas técnicamente correctas. Esto significaba que apenas se contaba con ejemplos negativos, imprescindibles para que la red aprendiera a reconocer una repetición incorrecta: una profundidad insuficiente, una inclinación excesiva del torso, un valgo de rodillas, una ejecución asimétrica o un ritmo inadecuado, ya fuera demasiado rápido o lento.

Tras muchas horas de búsqueda, en las que apenas se encontró material de sentadillas con ejecución defectuosa, se concluyó que no era razonable seguir invirtiendo tiempo en esa vía y que lo más sensato era grabar los ejemplos expresamente. No solo grabaciones propias, sino también de varias personas que se prestaron a colaborar: se les solicitaban ejecuciones de sentadilla que reprodujeran un defecto técnico determinado y las facilitaban. De este modo se consiguió un conjunto de datos bastante completo y equilibrado, capaz de entrenar la red no solo para la medición de los KPIs —el objetivo fundamental—, sino también para la detección de repeticiones y la segmentación de fases dentro de cada repetición.

5.4.3 Etiquetado de los ejemplos

Una vez reunido el primer dataset, surgió otro problema que iba a exigir muchas horas de trabajo: el etiquetado de los datos. No existía ninguna aplicación, modelo de inteligencia artificial ni herramienta que permitiera etiquetar de forma asistida horas de vídeo de sentadillas. Por ello se decidió desarrollar una herramienta de etiquetado propia en Python, apoyada en OpenCV, cuyo uso ya estaba integrado en la fase de preprocesado.

Esta herramienta permitía reproducir los vídeos y, simultáneamente, ir etiquetando las fases de cada repetición (de pie, bajando, abajo y subiendo) a medida que se visualizaban. La clave estaba en su funcionamiento: el vídeo se reproducía a una velocidad ajustable, normalmente a 0,5x, valor óptimo que permitía un etiquetado preciso sin alargar excesivamente el proceso. Cada fase se asignaba mediante las teclas numéricas del teclado (0 para «de pie», 1 para «bajando», y así sucesivamente). Además, disponía de controles para pausar, avanzar y retroceder en el vídeo, con la posibilidad de hacerlo fotograma a fotograma, lo que aportaba una gran precisión al marcar las transiciones entre fases.

Otro aspecto fundamental era que, cuando el sistema detectaba una secuencia completa de pie → de pie (es decir, una repetición íntegra), solicitaba introducir los KPIs correspondientes a esa repetición, lo que agilizaba notablemente el proceso. Todos los resultados se volcaban en un archivo .json que, posteriormente, serviría como fuente de ejemplos etiquetados para el entrenamiento de la red.

Implantación de imagen de la herramienta de etiquetado

5.4.4 Implantación de una metodología de trabajo

Más allá de la herramienta de etiquetado, ha sido determinante establecer una metodología de trabajo que permitiera seguir retroalimentando el conjunto de datos de forma continua. Una vez que la detección de fases alcanzó una fiabilidad alta, se buscó la manera de que cada vídeo probado en la aplicación, ya fuera para depuración o para evaluar la precisión del modelo, sirviera al mismo tiempo para enriquecer el dataset.

La solución adoptada funciona de la siguiente manera: cada vídeo procesado por la aplicación genera automáticamente un archivo .json con el resultado de la predicción. Basta entonces con revisar ese archivo, corregir únicamente los valores incorrectos y añadirlo al conjunto de datos general para incorporar el nuevo ejemplo. De este modo, cada vídeo de prueba adquiere un doble propósito tanto la validación del modelo como

su uso en el posterior reentrenamiento con ejemplos adicionales con un esfuerzo mínimo. Junto al .json, el sistema guarda además el vector de características en formato .npy, listo para su uso directo en el entrenamiento. Todo esto es configurable (el crear o no este archivo .json mediante una variable global).

```
# backend/app/services/video.py
EXPORT_TRAINING_JSON = True          # interruptor global del volcado para reentrenamiento
if EXPORT_TRAINING_JSON:
    # cada video analizado guarda su .json (prediccion) y su .npz (caracteristicas)
    ...                               # para enriquecer el dataset con un esfuerzo minimo
```

Este flujo de trabajo cierra el ciclo entre uso y mejora del modelo: cada interacción con la aplicación se convierte en una oportunidad para ampliar y refinar el conjunto de datos, lo que resulta especialmente valioso en un dominio donde, como se ha expuesto, el material de partida es escaso.

5.4.5 La técnica del espejo

Para lidiar con la escasez de datos se recurrió a una técnica de aumentado por espejo. Dado que una sentadilla reflejada horizontalmente sigue siendo, a todos los efectos técnicos, la misma sentadilla sus fases y todos sus indicadores (profundidad, torso, valgo, simetría y ritmo) son invariantes ante esa reflexión, cada ejemplo etiquetado puede transformarse en un segundo ejemplo igualmente válido sin necesidad de volver a etiquetarlo. El reflejo no se aplica sobre el vídeo, sino directamente sobre el vector de características ya procesado: se intercambian las columnas izquierda y derecha de las articulaciones (rodillas, caderas y tobillos), se niega la componente lateral del centro de sustentación, se recalcula la velocidad de la rodilla y, en el caso del valgo que es una magnitud con signo, las columnas se intercambian y se invierte su signo, ya que el eje medial-lateral del cuerpo se invierte en la imagen especular. De este modo se duplica el tamaño del conjunto de datos sin introducir ruido en las etiquetas, algo especialmente valioso teniendo en cuenta que la disponibilidad de datos era el principal cuello de botella del proyecto.

```
# backend/entrenamiento/mirror_utils.py
def mirror_npy_full(arr, fps=30.0):
    m = arr.copy().astype(np.float32); i = _IDX_FULL
    m[:, [i["L_knee"], i["R_knee"]]] = m[:, [i["R_knee"], i["L_knee"]]] # intercambia L <-> R
    m[:, i["sust_dx"]] = -m[:, i["sust_dx"]] # niega componente lateral
    m[:, i["L_valgus"]] = -old_R_valgus # valgo: swap + cambio de signo
    m[:, i["R_valgus"]] = -old_L_valgus
    return m
```

```
# backend/entrenamiento/training_xgb.py (uso como aumentado de datos)
raw_mir = mirror_npy_full(raw, fps=fps) # cada ejemplo se duplica con su reflejo
```

Conviene destacar que las características se diseñaron teniendo en cuenta que debían de ser compatibles con este aumentado: por ejemplo, el valgo se introduce finalmente en el modelo como valor absoluto, de modo que un ejemplo y su reflejo producen exactamente las mismas características. No debe confundirse esta técnica con otra idea, también basada en el espejo, que se consideró durante la investigación y se terminó descartando: reconstruir el lado ocluido del cuerpo reflejando el lado visible cuando la cámara solo capta con claridad un perfil. Esta segunda opción se rechazó porque fabricaría una simetría que puede no existir, corrompiendo precisamente uno de los indicadores que el sistema pretende medir como es la simetría, además de otros. Por ello, en lugar de inventar los puntos no visibles, el sistema los marca como datos ausentes, tal como se detalla en el tratamiento de la fiabilidad.

5.5 Diseño y evolución de los modelos

En este apartado se describe el proceso de evolución de los modelos de aprendizaje automático empleados en el sistema, así como la justificación de las principales decisiones tomadas a lo largo del desarrollo. El etiquetado, la construcción del dataset, el entrenamiento, el versionado y la puesta en producción de los modelos se recogen en el Anexo VI (Manual del programador).

5.5.1 Inicio del proceso

El planteamiento inicial consistía en emplear un único modelo capaz de realizar toda la tarea de análisis. La investigación previa permitió extraer una serie de conclusiones y evaluar distintos tipos de modelos, optándose finalmente por una red BiLSTM.

Modelo	Complejidad	Requerimiento de Datos	Ubicación de Ejecución	Puntos Fuertes / Observaciones
GRU	Baja	Bajos / Medios	Dispositivo Móvil	Detección rápida de las fases del ejercicio.
1D CNN	Baja	Bajos / Medios	Dispositivo Móvil	Excelente para clasificar series temporales simples.
LSTM	Media	Moderada	Dispositivo Móvil / Servidor	Alternativa sólida a GRU; un poco más de precisión general con un ligero aumento en el coste computacional.
BiLSTM	Media-Alta	Medios	Servidor (ideal)	Sobresale en el procesamiento a posteriori (post-entrenamiento), analizando contexto pasado y futuro de cada frame.
Transformer	Alta	Muy Altos	Demasiado pesado/complejo	Altísima capacidad de modelado, pero excesivo inicialmente.
ST-GCN	Máxima	Altos	Completamente excesivo para MVP	Analiza grafos espacio-temporales.

Tabla 3. Descripción de los distintos modelos comparados

Tras el estudio de alternativas desarrollado en el capítulo de Técnicas y herramientas, sintetizado en la Tabla 3, se optó por la BiLSTM como diseño inicial por ofrecer el mejor equilibrio entre capacidad de modelado y requerimientos de datos: su carácter bidireccional le permite analizar el contexto pasado y futuro de cada frame, algo ideal para el procesamiento a posteriori de la repetición, sin incurrir en los elevados requisitos de datos y cómputo de modelos como Transformer o ST-GCN.

5.5.2 Primeras versiones de la red BILSTM

Tras una cantidad considerable de pruebas, se resolvieron varios de los problemas iniciales, como el tratamiento de los valores ausentes, la eliminación de columnas redundantes o la incorporación de nuevas variables que facilitaran a la red una mejor interpretación de los datos. En la Tabla 2 se recoge la evolución de las primeras versiones del modelo.

Versión	Cambios introducidos	Resultado
V1–V2	Primeros entrenamientos con configuración básica y pocos ajustes	Línea base, resultados pobres
V3	Incorporación del centro de sustentación y reducción a las columnas estrictamente necesarias	Primer hito satisfactorio; conclusiones coherentes
V4–V8	Experimentos con el tratamiento de los valores ausentes (0s)	Fallidos; el conteo de repeticiones se degrada (V8 ni converge)
V9	Reentrenamiento con un mayor número de vídeos etiquetados	Salto de calidad; Buena profundidad fiable, pero problemas en ritmo y estabilidad
V10–V12	Introducción de velocidades, ajustes en la máscara de profundidad y mayor peso de los KPIs	Mejora en técnica general; profundidad inestable hasta corregir la máscara (V11)
V13	Etiquetado constante, normalización con máscara inteligente, filtro Savitzky-Golay y arquitectura de doble salida	Reducción notable del error y mejor generalización

Tabla 4. Evolución de las versiones del modelo BILSTM

Aunque en un primer momento todo parecía funcionar correctamente, el verdadero problema surgió al ampliar las pruebas con ejemplos de sentadillas incorrectas. En estos casos el rendimiento del modelo se degradaba notablemente, lo que evidenciaba que el resultado distaba de ser óptimo. Esto motivó un análisis en profundidad de las causas del fallo con el fin de abordar un cambio sustancial en el planteamiento.

5.5.3 Cambio de escala y uso de XGBoost

El análisis reveló dos problemas fundamentales. El primero era la escasez de vídeos de ejecuciones incorrectas; aun después de recopilar nuevos ejemplos, el desbalanceo del

conjunto de datos con un número notablemente mayor de ejecuciones correctas que incorrectas seguía impidiendo obtener un resultado óptimo.

El segundo problema era la subjetividad implícita en el etiquetado manual. Inicialmente, cada KPI se etiquetaba siguiendo una escala del 0 al 10, sin más apoyo que la herramienta de etiquetado desarrollada, que asistía en el proceso, pero no en la elección del valor concreto de cada KPI. Esto resultaba problemático: dos repeticiones muy similares podían recibir valores distintos (por ejemplo, un 7 y un 8 en un mismo KPI), debido a la dificultad de mantener un criterio exacto y constante a lo largo de los cientos de repeticiones analizadas.

Para abordar el primer problema se propuso emplear un modelo XGBoost para la extracción de los KPIs por repetición, dado que este tipo de modelo requiere una menor cantidad de datos para su entrenamiento. Siguiendo su consejo, se separó el proceso en dos etapas: la segmentación de las fases de la sentadilla mediante la red BiLSTM y la extracción de los KPIs por repetición mediante XGBoost. Asimismo, se incorporó la detección del perfil de la toma, puesto que, como se ha señalado anteriormente, resulta físicamente imposible analizar el valgo de rodilla o la simetría desde una vista completamente lateral.

Para resolver el problema de la subjetividad en el etiquetado se optó por simplificarlo. En lugar de una nota exacta del 0 al 10, cada KPI pasó a clasificarse en tres categorías: erróneo, aceptable y excelente. Esta simplificación no solo agilizó el etiquetado, sino que permitió establecer un criterio común y reproducible, de modo que repeticiones similares reciben ahora la misma etiqueta en un número mucho mayor de casos. Además, esta escala resulta más útil de cara al usuario, ya que el objetivo no es tanto conocer una puntuación precisa como saber si la ejecución es excelente (sin aspectos que mejorar), aceptable (correcta pero mejorable) o errónea (con una técnica que es de vital importancia corregir para evitar lesiones).

5.5.4 XGBoost y BiLSTM

Llegados a este punto, surge una pregunta razonable: ¿por qué no emplear XGBoost para todo el análisis?

La respuesta reside en el funcionamiento de cada modelo a bajo nivel. El carácter bidireccional de la red BiLSTM resulta determinante para la segmentación de fases, ya que permite etiquetar cada fotograma considerando tanto el contexto pasado como el

futuro de la secuencia, lo que se traduce en transiciones de fase más coherentes y en un recuento de repeticiones más robusto. La arquitectura de la red BiLSTM se define en la función `build_model()`:

```
# backend/entrenamiento/training_phases.py

def build_model(T_max):
    inp = layers.Input(shape=(T_max, N_FEATURES))
    x = layers.Masking(mask_value=0.0)(inp)
    x = layers.Bidirectional(layers.LSTM(64, return_sequences=True))(x) # ignora el padding # contexto
    pasado+futuro
    x = layers.Dropout(0.3)(x)
    x = layers.Bidirectional(layers.LSTM(32, return_sequences=True))(x)
    x = layers.Dropout(0.3)(x)
    out = layers.TimeDistributed(layers.Dense(4, activation="softmax"))(x) # 1 fase por frame
    return Model(inp, out, name="BiLSTM_phases")
```

XGBoost, en cambio, es un modelo basado en árboles de decisión que no considera más que el instante actual: resulta imposible determinar si un sujeto está subiendo o bajando, observando un fotograma aislado, pues se necesita un contexto temporal previo que la BiLSTM sí captura y XGBoost no.

```
# backend/entrenamiento/training_xgb.py

DEFAULT_PARAMS = dict(n_estimators=300, max_depth=4, learning_rate=0.05,
                      subsample=0.8, colsample_bytree=0.8)

# Un modelo XGBoost por KPI, validado con StratifiedGroupKFold (agrupado por video)
clf = XGBClassifier(eval_metric=eval_metric, random_state=42, n_jobs=-1, **best_params)
clf.fit(X_fit, y_fit, sample_weight=w_fit)
```

Sin embargo, la extracción de los KPIs no requiere ese contexto. Si al modelo le llega, por ejemplo, el instante de máxima profundidad e inclinación del torso, no necesita información adicional para emitir un resultado. Por este motivo se emplea XGBoost en esta tarea: es un modelo más simple que ofrece un resultado igual o mejor que la BiLSTM para la obtención de KPIs, con la ventaja decisiva de requerir un conjunto de datos menor, que constituye el principal cuello de botella del proyecto.



Figura 6. Arquitectura de dos etapas: BiLSTM + XGBoost

5.5.5 *Evaluación del sistema final*

Una vez definida la arquitectura de dos etapas, se evaluó el rendimiento de cada modelo sobre datos no empleados en su entrenamiento. La red BiLSTM se evaluó sobre el conjunto de validación, comparando la fase predicha en cada fotograma con la etiqueta real; el modelo XGBoost se evaluó mediante validación cruzada agrupada por vídeo, tal y como se justificó anteriormente.

En la segmentación de fases, la BiLSTM alcanzó una exactitud global en torno al 80 % a nivel de fotograma. El comportamiento por fase es coherente con la naturaleza del problema: las fases de descenso y ascenso, mayoritarias y de mayor duración, se reconocen con una F1 cercana a 0,83, mientras que la fase de máxima profundidad, minoritaria y muy breve, es la más difícil de delimitar (F1 en torno a 0,61), un resultado esperable dada su escasa representación en los datos.

Conviene matizar estos porcentajes. La exactitud en torno al 80 % se refiere al nivel de fotograma es decir, de cada 100 fotogramas, unos 80 se clasifican correctamente, pero ese no es el objetivo final: las fases solo sirven para delimitar las repeticiones, y a nivel de repetición completa el sistema acierta casi siempre. La aparente limitación se explica porque las transiciones entre fases se extienden a lo largo de varios fotogramas, de modo que en esos límites resulta intrínsecamente ambiguo determinar la fase exacta de un fotograma aislado; el error se concentra en esas fronteras y no en la identificación de las repeticiones.

En la evaluación de los KPIs por repetición, los resultados varían según el indicador. La profundidad y la inclinación del torso obtienen exactitudes en torno a 0,65 y 0,72 respectivamente en el plano frontal. Los indicadores binarios de valgo y simetría presentan exactitudes aparentemente altas (0,86 y 0,84), el modelo acierta con frecuencia la clase mayoritaria, por lo que la medida F1 resulta más informativa que la exactitud para estos casos. El ritmo es el indicador con peor desempeño, especialmente en el plano lateral, lo que se atribuye a la dificultad de capturar la cadencia del movimiento con la cantidad de datos disponible.

En conjunto, estos resultados reflejan un sistema funcional y consciente de sus limitaciones: sólido en las tareas estructurales como el conteo de repeticiones y la segmentación de fases y más modesto en los indicadores dinámicos, más dependientes de la cantidad y variedad de datos.

El análisis de los errores por clase confirma estas conclusiones: la confusión se concentra entre fases o categorías contiguas, rara vez entre extremos.

Cabe aplicar a los KPIs la misma contextualización que a la segmentación de fases. Aunque algunos valores puedan parecer bajos, conviene interpretarlos con cautela: buena parte del error se concentra en las repeticiones que caen en la frontera entre dos categorías es decir, entre excelente y aceptable, o entre aceptable y errónea, casos ambiguos que resultan difíciles de dirimir incluso en el etiquetado manual. Precisamente por ello se simplificó la escala original de 0–10 a tres categorías, en busca de una distinción más nítida, aun así sigue siendo difícil el etiquetado manual incluso con este cambio de escala. En conjunto, y pese a los porcentajes, el sistema distingue con claridad una repetición correcta de una incorrecta, y sus dificultades se limitan a esos casos fronterizos.

Los resultados de la evaluación se resumen en las dos tablas siguientes.

Fase	Precisión	Recall	F1
De pie	0,83	0,74	0,78
Bajando	0,82	0,86	0,84
Abajo	0,54	0,70	0,61
Subiendo	0,83	0,83	0,83

Tabla 5. Resultados de la evaluación de predicción de fase.

KPI	Frontal	Lateral
Profundidad	0,65	0,59
Torso	0,72	0,70
Valgo	0,86	—
Simetría	0,84	—
Ritmo	0,60	0,35

Tabla 6. Resultados de la evaluación de KPIs

5.6 Backend y procesamiento asíncrono

El backend constituye el lado servidor del sistema: recibe los vídeos desde la aplicación, ejecuta sobre ellos el pipeline de análisis descrito en los apartados anteriores, persiste los resultados y los expone de nuevo a la aplicación. Se ha construido con FastAPI, cuya elección y pila tecnológica se justificaron en el capítulo de Técnicas y herramientas; en este apartado se describe su diseño interno y, en particular, cómo resuelve el principal reto de la parte servidor: que el análisis de un vídeo es una tarea lenta que no puede resolverse dentro de una única petición HTTP.

5.6.1 *Diseño de la API REST*

La API se organiza en routers, cada uno responsable de un área funcional del sistema y agrupados bajo el prefijo versionado `/api/v1`: la autenticación (auth), las sesiones de análisis (sessions), la analítica de la evolución (analytics), el registro de molestias e incidencias (checkins, incidents), el bienestar (wellness) y las recomendaciones (recommendations). Esta separación mantiene cada conjunto de endpoints aislado y facilita el mantenimiento, además, gracias a esta separación modular, la escalabilidad y la facilidad de implementar mejoras o nuevas funcionalidades se convierte en una tarea sencilla, otro punto clave que nos aporta esta modularidad es el mantenimiento de la propia app.

De FastAPI se aprovechan dos características de forma simultánea. La primera es la generación automática de documentación interactiva de la API a partir del propio código y de los modelos de datos. La segunda es la inyección de dependencias mediante Depends, empleada para dos tareas que se repiten en casi todos los endpoints: obtener una sesión de base de datos (`get_db`) y resolver el usuario autenticado a partir de su token (`get_current_user`). De este modo, la lógica de cada endpoint queda libre de código repetitivo de infraestructura.

La autenticación se implementa con JSON Web Tokens. Al iniciar sesión, el servidor emite dos tokens: uno de acceso, de vida corta, y uno de refresco, de vida larga; este último permite renovar el de acceso a través de un endpoint específico sin obligar al usuario a introducir de nuevo sus credenciales. La gestión de esa renovación desde el lado del cliente se detalla en el apartado de la aplicación móvil.

El diseño de los endpoints y de sus formatos de petición y respuesta se detalla en el Anexo II (Análisis y diseño del sistema), mientras que su contrato definitivo y la

documentación OpenAPI generada para el desarrollador se recogen en el Anexo VI (Manual del programador).

5.6.2 *Procesamiento asíncrono en el servidor*

El análisis de un vídeo como ya ha sido mencionado corresponde con la estimación de pose fotograma a fotograma, cálculo de las métricas biomecánicas e inferencia de los dos modelos, esto es una tarea costosa que puede prolongarse durante un tiempo considerable. Resolverla dentro de la propia petición de subida obligaría al cliente a esperar una respuesta durante todo ese proceso, con el riesgo de agotar los tiempos de espera de la red. Para evitarlo, el sistema separa la recepción del vídeo de su análisis mediante una máquina de estados.

Cuando el usuario sube un vídeo, a través del endpoint POST /sessions/upload, el servidor guarda el archivo en disco, crea un registro de sesión en la base de datos con estado queued (en cola) y, en lugar de procesarlo en ese instante, delega el análisis a una tarea en segundo plano mediante el mecanismo BackgroundTasks de FastAPI. A continuación, responde de inmediato con un código 202 (Accepted), que indica que la petición se ha aceptado pero su procesamiento continúa. Así, el cliente recibe respuesta en milisegundos. Lo que permite al usuario seguir navegando por la aplicación si lo desea sin necesidad de esperar a que el análisis concluya.

La tarea en segundo plano, la función process_video, es la que ejecuta el pipeline completo descrito en los apartados anteriores. Su primer paso es marcar la sesión como processing y, al finalizar correctamente, como completed; si se produce cualquier error durante el análisis, la marca como failed. Estos cuatro estados: queued, processing, completed y failed, constituyen la máquina de estados que la aplicación consulta periódicamente para saber cuándo el resultado está disponible. Ese sondeo, desde el lado del cliente, se describe en el apartado 5.7 Aplicación móvil.

Un detalle técnico relevante es que la tarea en segundo plano abre su propia sesión de base de datos, independiente de la de la petición original. El motivo es que, cuando el análisis comienza a ejecutarse, la petición de subida ya ha concluido y su sesión de base de datos se ha cerrado, por lo que la tarea no puede reutilizarla y necesita una propia.

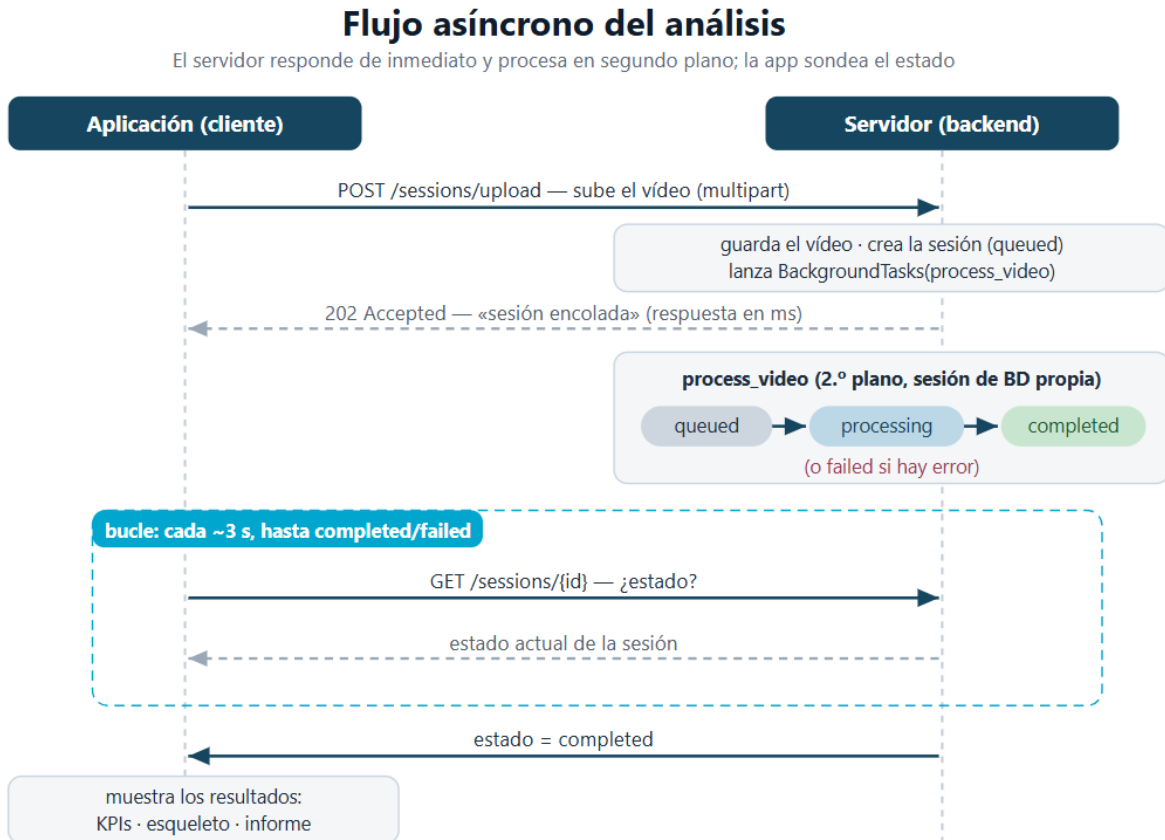


Figura 7. Flujo asíncrono en el servidor

5.6.3 Persistencia y modelo de datos

La persistencia se gestiona con el ORM SQLAlchemy sobre una base de datos MariaDB, lo que permite manipular los datos como objetos de Python en lugar de escribir SQL directamente. El modelo de datos se articula en torno a unas pocas entidades principales:

- Usuario: los datos de la cuenta, las credenciales y el perfil del usuario.
- Sesión: cada vídeo analizado, con su estado, los parámetros de la toma (peso y esfuerzo percibido) y las referencias a los ficheros generados.
- Resultado por repetición: una fila por cada repetición detectada, con la puntuación de cada KPI. Los campos de valgo y simetría quedan vacíos (NULL) cuando el vídeo se ha grabado en plano lateral, donde, como se explicó en la detección del perfil, esos indicadores no son medibles.
- Registro de molestias e incidencias: el seguimiento del estado físico del usuario a lo largo del tiempo.

Una sesión agrupa varios resultados de repetición, de modo que la relación entre ambas entidades es de uno a varios.

El modelo lógico y físico de la base de datos se detalla en el Anexo II (Diseño de datos).

Por último, los ficheros que genera el análisis como son: el informe en PDF y el vídeo con el esqueleto superpuesto; no se almacenan en la base de datos, sino en disco, y se sirven a la aplicación como ficheros estáticos: el informe mediante el montaje de archivos estáticos de FastAPI y el vídeo de esqueleto a través de un endpoint específico que lo entrega bajo demanda. La ubicación de estos ficheros en volúmenes persistentes se detalla en el apartado de Despliegue.

5.7 Aplicación móvil

La aplicación móvil es la cara visible del sistema, es el medio a través del cual el usuario graba o sube sus sentadillas, recibe la evaluación técnica y realiza el seguimiento de su evolución. Se ha desarrollado de forma nativa para Android, en lenguaje Kotlin, priorizando una arquitectura limpia y mantenible que permitiera incorporar nuevas funcionalidades sin comprometer la estabilidad del conjunto.

5.7.1 Tecnologías y elección de Jetpack Compose

Para la construcción de la interfaz se optó por Jetpack Compose, el sistema moderno de Android para construir pantallas reactivas usando únicamente código, en lugar del sistema clásico basado en XML. La principal razón de esta decisión es que Compose describe la interfaz como una función del estado: en lugar de modificar manualmente los componentes en cada cambio, se declara cómo debe verse la pantalla para un estado dado y el framework se encarga de redibujar únicamente lo necesario.

Sobre Compose se utiliza Material 3 como sistema de diseño, junto con un tema propio (paleta de azul marino y cian, tipografías y formas personalizadas) que otorga a la aplicación de una identidad visual característica en todas sus pantallas. La aplicación se compone una serie de pantallas que cubren el ciclo completo: autenticación, captura y subida de vídeo, resultados, seguimiento del bienestar, recomendaciones e historial.

5.7.2 *Arquitectura: Clean Architecture y patrón MVVM*

La aplicación se estructura siguiendo los principios de arquitectura limpia, organizada en capas con responsabilidades bien delimitadas y dependencias dirigidas siempre hacia el interior (las capas externas dependen de las internas, nunca al revés):

- Capa de dominio (domain): contiene la lógica de negocio pura, independiente de Android. Define los modelos del dominio, las interfaces de los repositorios y los casos de uso, cada uno de los cuales encapsula una operación concreta del sistema (iniciar sesión, obtener las recomendaciones, exportar el informe, etc.). Esta capa no conoce ni la interfaz ni el origen de los datos.
- Capa de datos (data): implementa los repositorios definidos por el dominio. Se encarga de la comunicación con la API REST mediante objetos de transferencia (DTO) y de su conversión a los modelos del dominio, de modo que el resto de la aplicación nunca manipula directamente la representación del servidor.
- Capa de presentación (presentation): contiene las pantallas en Compose y sus ViewModels, siguiendo el patrón MVVM (Model–View–ViewModel). Cada pantalla observa un estado expuesto por su ViewModel y reacciona a sus cambios, sin contener lógica de negocio.
- Capa transversal (core): agrupa los aspectos comunes a toda la aplicación: la inyección de dependencias, la configuración de red, la seguridad, la gestión de la sesión y las utilidades compartidas.

La inyección de dependencias se gestiona con Hilt, que proporciona de forma automática las instancias necesarias (servicios de red, repositorios, gestores de sesión) allí donde se requieren. Esto desacopla los componentes, facilita las pruebas y evita la creación manual de objetos. El patrón MVVM, combinado con esta separación en capas, permite que cada parte del sistema evolucione de forma independiente: un cambio en la API solo afecta a la capa de datos, y un rediseño de una pantalla no altera la lógica de negocio.

La organización del código del cliente por capas se documenta completamente en el Anexo VI (Manual del programador).

5.7.3 *Gestión del análisis asíncrono*

El análisis de un vídeo es un proceso costoso que puede tardar bastante en el servidor, por lo que no podía resolverse en una única petición síncrona. Para gestionarlo se diseñó un flujo asíncrono de sondeo:

1. El usuario sube el vídeo mediante una petición multipart. El servidor responde de inmediato (sin esperar al análisis) confirmando que la sesión se ha encolado.
2. La aplicación consulta entonces periódicamente el estado de la sesión, cada pocos segundos, hasta que el servidor lo marca como completado o fallido.
3. Mientras tanto, la interfaz muestra una pantalla de carga con mensajes informativos.

Un detalle de experiencia de usuario relevante es que, durante la espera del análisis, la aplicación mantiene la pantalla encendida, evitando que el dispositivo entre en reposo e interrumpa la conexión en mitad del proceso.

5.7.4 *Comunicación con el backend y gestión de la sesión*

La comunicación con el servidor se realiza mediante Retrofit sobre OkHttp, mientras que la conversión entre JSON y objetos Kotlin la efectúa Gson. La dirección del servidor no está fijada en el código, sino que se inyecta en tiempo de compilación a través de BuildConfig, lo que permite alternar entre el entorno local de desarrollo y el servidor sin modificar la lógica.

El aspecto más relevante de esta capa es la gestión de la autenticación mediante JWT. El sistema emplea dos tokens: un token de acceso de vida corta y un token de refresco de vida larga. Para que el usuario no tenga que volver a iniciar sesión continuamente, se implementaron dos interceptores en la capa de red:

- Un interceptor de autenticación que añade automáticamente el token de acceso a cada petición saliente.
- Un autenticador que detecta las respuestas de tipo 401 (no autorizado) es decir, cuando el token de acceso ha caducado, solicita uno nuevo de forma transparente usando el token de refresco y reintenta la petición original sin que el usuario perciba nada.

Los tokens y las preferencias del usuario se almacenan localmente mediante DataStore, la solución recomendada por Android para la persistencia de datos clave-valor. Un gestor de sesión centraliza el estado de autenticación y permite que la

aplicación decida, al iniciar, si dirigir al usuario a la pantalla de inicio de sesión o directamente a la principal.

La renovación del token de acceso ante un error 401 la realiza de forma transparente el `TokenAuthenticator`, que reintenta la petición original con el nuevo token:

```
# frontend - core/network/TokenAuthenticator.kt
override fun authenticate(route: Route?, response: Response): Request? {
    if (response.request.url.encodedPath.contains("/auth/refresh")) { expireSession(); return null }
    val refreshToken = runBlocking { datastore.getRefreshToken().firstOrNull() } ?: return null
    val newTokens = callRefreshEndpoint(refreshToken) ?: return null // renueva el token
    runBlocking { datastore.saveTokens(newTokens.first, newTokens.second) }
    return response.request.newBuilder() // reintenta la
    petición
        .header("Authorization", "Bearer ${newTokens.first}").build()
}
```

5.7.5 Visualización de resultados, seguimiento y notificaciones

Una vez completado el análisis, la pantalla de resultados presenta la evaluación: la puntuación de cada KPI, las medias por métrica, la comparación con la sesión anterior y un indicador de fatiga intra-sesión. La aplicación integra un reproductor de vídeo tradicional para mostrar el esqueleto superpuesto y permite exportar el informe de la sesión en PDF.

Más allá del análisis puntual, la aplicación incorpora un módulo de seguimiento del usuario: registro del estado físico, evolución del bienestar mediante gráficas, registro de molestias o incidencias y recomendaciones personalizadas. Para aumentar la eficiencia del seguimiento sin requerir que la aplicación esté abierta, se usa `WorkManager`, la solución de Android para la ejecución de tareas en segundo plano de forma fiable. Mediante esta herramienta se programan notificaciones locales: recordatorios para registrar las molestias tras el entrenamiento y un resumen semanal de la evolución, estas notificaciones son configurables pudiendo evitarlas. Estas tareas se ejecutan respetando las restricciones del sistema operativo y sobreviven a reinicios del dispositivo, garantizando que el seguimiento se mantenga activo a lo largo del tiempo.

Las distintas pantallas de la aplicación se muestran y documentan en su totalidad en el Anexo V (Manual de usuario). A nivel técnico, el análisis se gestiona de forma asíncrona mediante sondeo y la sesión se mantiene con renovación transparente de los tokens.

El sondeo del estado del análisis se implementa en pollSessionResults():

```
# frontend - presentation/results/ResultsViewModel.kt
private fun pollSessionResults() = viewModelScope.launch {
    while (true) {
        when (val result = getSessionDetailUseCase(sessionId)) {
            is Resource.Success -> when {
                result.data.status == "failed" -> return@launch // error
                result.data.isCompleted -> return@launch // mostrar KPIs
                else -> delay(3_000L) // sigue procesando
            }
            is Resource.Error -> return@launch
            is Resource.Loading -> delay(3_000L)
        }
    }
}
```

5.8 Seguimiento del bienestar y prevención de lesiones

Más allá del análisis técnico de cada repetición, el sistema incorpora un módulo de seguimiento constante del usuario cuyo propósito es estimar el estado de forma del usuario, anticipar situaciones de riesgo y orientar sus decisiones de entrenamiento a lo largo del tiempo. Este módulo reside fundamentalmente en el backend concretamente en los servicios de readiness y de recomendaciones y en el conjunto de endpoints de bienestar y se expone a la aplicación a través de la API. Conviene subrayar desde el inicio que todas sus salidas son de carácter orientativo y en ningún caso constituyen un diagnóstico médico, advertencia que se traslada de forma explícita al usuario.

5.8.1 Entradas del módulo: esfuerzo percibido (Borg) y dolor (EVA)

El módulo se nutre de dos valoraciones subjetivas que aporta el propio usuario y que complementan las medidas objetivas extraídas del vídeo. La primera es el esfuerzo percibido recogido mediante la escala de Borg [Borg, 1982] en su versión de 0 a 10. El usuario introduce este valor al subir cada sesión, indicando cómo de exigente le ha resultado el conjunto de repeticiones. El esfuerzo percibido es un indicador ampliamente utilizado en las ciencias del deporte para cuantificar la denominada carga interna del entrenamiento, aquella que depende de la respuesta del individuo y no únicamente del peso desplazado.

La segunda valoración es el dolor, registrado mediante la Escala Visual Analógica (EVA) [Huskisson, 1974], igualmente de 0 a 10. A diferencia del esfuerzo, la EVA no se recoge durante la sesión, sino en los check-ins posteriores, a las veinticuatro y a las cuarenta y ocho horas del entrenamiento, intervalos en los que suele manifestarse el dolor muscular de aparición tardía. En cada check-in el usuario indica, además de la intensidad,

la zona corporal afectada, lo que permite distinguir una molestia puntual y transitoria de un dolor persistente y localizado.

5.8.2 Estimación del readiness

A partir de las valoraciones anteriores, junto con la carga de entrenamiento y la calidad técnica, el sistema calcula un indicador global de disposición para el entrenamiento, denominado readiness (en la aplicación mostrado en español bajo el nombre de estado físico) y representado al usuario como un semáforo de tres estados. El cálculo se implementa en la función `calculate_readiness` y combina cuatro dimensiones, cada una normalizada en una escala de 0 a 100:

- **Fatiga aguda.** Acumula la carga de las sesiones de los últimos catorce días ponderada por un decaimiento exponencial, de modo que el peso de cada sesión disminuye a medida que se aleja en el tiempo, con una vida media de tres días. La carga de una sesión se estima como el producto del peso movido, el número de repeticiones y el esfuerzo percibido; cuando la sentadilla se ejecuta sin carga externa se emplea el peso corporal del usuario. Una acumulación elevada en pocos días se traduce en una fatiga alta y, por tanto, en un readiness reducido.
- **Carga relativa (ACWR).** Es el cociente entre la carga aguda —la de los últimos siete días— y la carga crónica —el promedio semanal de las últimas cuatro semanas—, conocido en la literatura como *Acute:Chronic Workload Ratio*. Siguiendo la evidencia sintetizada por Gabbett [Gabbett, 2016], se considera óptima una franja entre 0,8 y 1,3: por debajo señala una carga insuficiente y por encima un incremento demasiado brusco que eleva el riesgo de lesión, penalizándose la puntuación de forma progresiva a partir de 1,3.
- **Dolor residual.** Se deriva del mayor valor de EVA registrado en los últimos catorce días, atenuado también por un decaimiento exponencial con una vida media de cuatro días que refleja la recuperación progresiva. Un dolor reciente e intenso rebaja de forma acusada el readiness.
- **Técnica reciente.** Es la media de la puntuación técnica de las últimas cinco sesiones, bajo la premisa de que una técnica que se degrada suele ser síntoma de fatiga acumulada o de un riesgo creciente.

El peso de cada dimensión no es fijo, sino que se adapta a la madurez del histórico del usuario para evitar emitir juicios poco fiables cuando aún se dispone de pocos datos.

Se distinguen tres etapas: en el arranque (menos de siete días de actividad o menos de tres sesiones) el readiness se basa exclusivamente en la fatiga aguda; en una etapa intermedia (hasta cuatro semanas) se combinan la fatiga (50 %), el dolor (30 %) y la técnica (20 %); y en la etapa consolidada (a partir de cuatro semanas, siempre que exista carga crónica suficiente) se incorpora la carga relativa, con una distribución del 30 % para la fatiga, el 25 % para el ACWR, el 25 % para el dolor y el 20 % para la técnica.

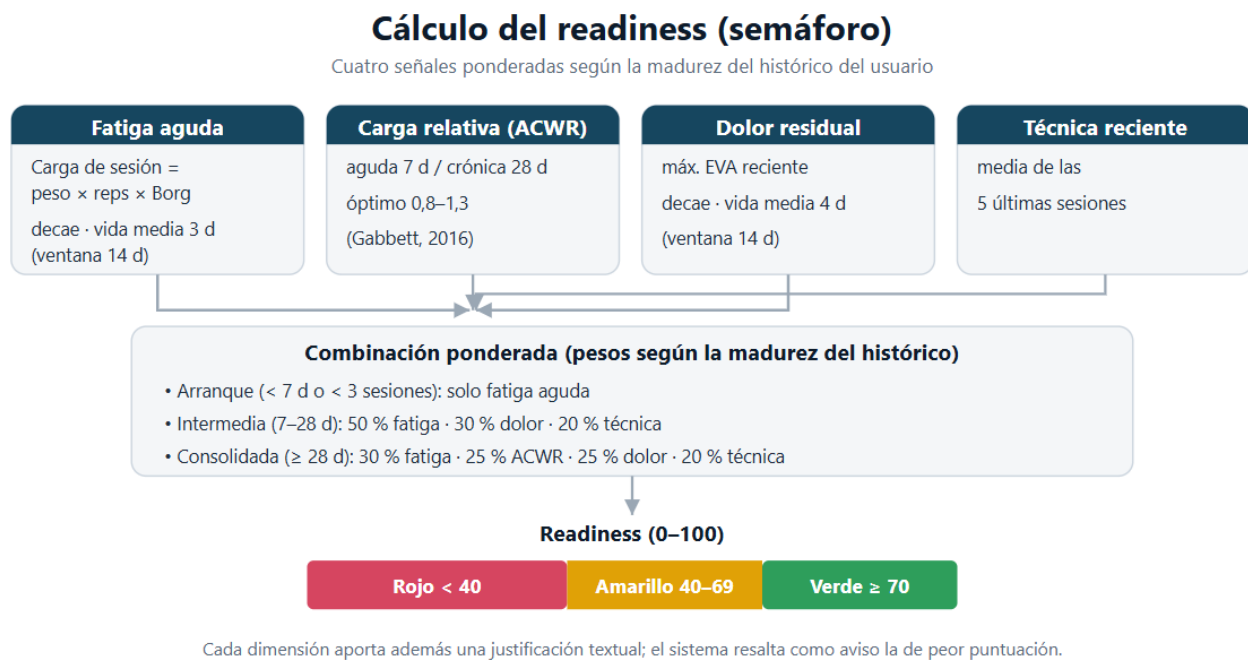


Figura 8. Cálculo del semáforo

Cada componente lleva asociada una justificación textual generada de forma automática (por ejemplo, «carga un 40 % por encima de tu media; vigila la fatiga»), y el sistema destaca como aviso principal la dimensión con peor puntuación. De este modo el usuario no recibe únicamente un valor numérico, sino también la razón concreta que lo explica.

5.8.3 Motor de recomendaciones

Sobre el estado calculado opera un motor de recomendaciones, implementado en la función `evaluate_recommendations`. No se trata de un modelo generativo, sino de un catálogo curado de recomendaciones asociadas a un conjunto de disparadores deterministas que se evalúan sobre el estado actual del usuario. Cada recomendación pertenece a una de estas cuatro categorías: técnica, carga, recuperación y progresión, e incorpora un texto explicativo y un vídeo educativo de apoyo.

Los disparadores combinan las salidas del análisis técnico y las del módulo de bienestar. Así, una puntuación de profundidad, torso o valgo inferior a 5 sobre 10 activa la recomendación técnica correspondiente; un ACWR superior a 1,5 activa una pauta de reducción de la carga; un valor de EVA superior a 6 en la última semana propone medidas de recuperación; más de diez días sin actividad sugiere una reincorporación gradual; y un readiness inferior a 40 recomienda una semana de descarga. En sentido opuesto, la combinación de un esfuerzo percibido bajo (Borg medio inferior a 3) con una técnica sólida (igual o superior a 7) habilita una recomendación de progresión. Las recomendaciones que resultan activas se ordenan por prioridad antes de presentarse al usuario.



Figura 9. Motor de recomendaciones

5.8.4 Análisis de factores previos a una lesión

Cuando el usuario registra una incidencia, puede ser una molestia, contractura, dolor agudo o lesión, estará localizada en una zona corporal concreta y con una fecha de inicio, el sistema ofrece un análisis de los factores observados en los días que la precedieron. Para ello compara la ventana de los catorce días anteriores al inicio de la incidencia con la línea base del propio usuario, esto es, con el resto de su histórico, y cuantifica la desviación porcentual de varias métricas: la carga media por sesión, la frecuencia de

entrenamiento, la puntuación técnica, el esfuerzo percibido y el dolor previo. Cada factor se clasifica, según la magnitud de su desviación, en bajo, moderado o alto.

El enfoque adoptado es deliberadamente prudente: el sistema no afirma en ningún momento una relación de causalidad, sino que se limita a describir qué métricas se apartaron del comportamiento habitual del usuario antes de la incidencia. Determinar la causa real de una lesión excede con mucho las posibilidades de un análisis basado únicamente en estos datos, y presentarlo como un diagnóstico sería tan arriesgado como deshonesto. Esta cautela es coherente con el carácter de herramienta de apoyo que define al sistema en su conjunto.

5.8.5 Validación retrospectiva del semáforo

Por último, el módulo incorpora un mecanismo de validación retrospectiva que permite estimar la capacidad predictiva del propio semáforo. Para cada incidencia registrada, el sistema recalcula el readiness que el usuario presentaba en los siete días previos a su inicio y lo compara con el de la semana anterior, si se aprecia un descenso de al menos diez puntos, el caso se contabiliza como una alerta temprana. Agregando este análisis sobre todas las incidencias se obtiene la proporción de ocasiones en las que el readiness anticipó un deterioro antes de la aparición del problema. El reducido número de incidencias que un único usuario puede registrar limita, lógicamente, la solidez estadística de esta medida; no obstante, constituye un primer indicio cuantitativo del valor del indicador como herramienta de prevención y deja planteada una vía de evaluación que cobraría pleno sentido con una base de usuarios amplia.

5.8.6 Exportación en PDF

La información de cada sesión, junto con toda la información generada por este módulo (puntuaciones técnicas, récords personales, desglose del readiness con sus componentes, recomendaciones activas, histórico de incidencias y gráficas de evolución con los periodos de lesión resaltados), se consolida en un informe profesional en formato PDF que el usuario puede exportar y, llegado el caso, compartir con un profesional sanitario o deportivo, lo que refuerza una vez más el carácter de apoyo, y no de sustituto, del sistema.

5.9 Despliegue

5.9.1 Despliegue del sistema

El despliegue del sistema parte de su arquitectura, que responde a un modelo cliente-servidor clásico. El cliente reside en un dispositivo móvil Android, mientras que el servidor concentra el procesamiento de vídeo, la inferencia de los modelos y la gestión de la base de datos. La comunicación entre ambos se realiza mediante peticiones REST sobre HTTPS.

5.9.2 Despliegue del servidor

El servidor se aloja en una máquina con sistema operativo Ubuntu 24, cedida por la Universidad de Salamanca para alojar este trabajo, accesible de forma remota mediante SSH y publicada bajo el dominio público:

`prodiasv07.fis.usal.es.`

Para el despliegue se ha optado por Docker, encapsulando la aplicación y todas sus dependencias en contenedores. Esta decisión elimina los problemas derivados de la heterogeneidad de entornos (diferencias de sistema operativo, versiones de librerías o dependencias incompatibles), garantizando que el sistema se ejecute de forma idéntica con independencia de la máquina sobre la que se despliegue, además de facilitar su portabilidad y mantenimiento.

El sistema se estructura en tres contenedores orquestados mediante docker-compose, cada uno con una responsabilidad bien definida:

Servicio	Imagen	Rol
nginx	nginx:alpine	Proxy inverso, terminación HTTPS y servicio de ficheros estáticos
api	Imagen propia (FastAPI)	Lógica de negocio y pipeline de aprendizaje automático
db	mariadb:11	Base de datos

Tabla 7. Contenedores Docker

Estos servicios se comunican a través de una red interna de Docker, de modo que cada uno es accesible para los demás mediante su nombre de servicio en lugar de por dirección IP. Únicamente el contenedor de Nginx publica puertos hacia el exterior, lo que aísla tanto la API como la base de datos de accesos directos desde internet.

Delante de la aplicación se sitúa un proxy inverso Nginx, una decisión que aporta varias ventajas: se encarga de la terminación TLS/HTTPS (los certificados residen en Nginx y no en la aplicación), redirige el tráfico del puerto 80 al 443, sirve de forma eficiente los ficheros estáticos generados (informes y vídeos de esqueleto) y gestiona las particularidades de este sistema, como las subidas de vídeo de gran tamaño y los tiempos de espera prolongados propios del análisis. Además, evita exponer directamente el servidor de aplicaciones a internet.

Internamente, FastAPI se ejecuta sobre Uvicorn con varios procesos (workers), lo que aporta paralelismo y reinicio ante caídas. Un aspecto relevante en este punto es que el pipeline carga en memoria modelos pesados (TensorFlow, MediaPipe y XGBoost) una vez por cada worker, por lo que un número elevado de estos multiplicaría el consumo de memoria. Dado que se trata de un servidor de un único usuario y que el análisis es intensivo en CPU, se optó por un número reducido de workers, priorizando la estabilidad frente a una concurrencia que no resultaba necesaria.

En cuanto a la persistencia, se distinguen dos tipos de datos que deben sobrevivir a los reinicios y recreaciones de los contenedores. Por un lado, la base de datos MariaDB emplea un volumen Docker con nombre. Por otro, los ficheros generados por la aplicación como son los vídeos subidos, informes y vídeos de esqueleto, se almacenan igualmente en volúmenes, evitando su pérdida al recrear el contenedor de la API. Los modelos entrenados, al ser de solo lectura, se copian directamente dentro de la imagen durante su construcción y no requieren volumen.

Respecto a la seguridad, se han adoptado varias medidas básicas pero relevantes. El acceso se realiza íntegramente sobre HTTPS, con la terminación TLS gestionada por Nginx. El cortafuegos del servidor restringe el acceso a los únicos puertos necesarios (SSH, HTTP y HTTPS), y la base de datos no expone su puerto al exterior, siendo accesible exclusivamente desde la red interna de Docker. Finalmente, los datos sensibles, como la clave de firma de los tokens JWT y las credenciales de la base de datos, se gestionan mediante variables de entorno externas al código y al control de versiones.

El análisis de riesgos, las salvaguardas, los aspectos legales (RGPD, EIPD, Reglamento de IA) y la trazabilidad requisitos→controles se desarrollan en el Anexo IV (Plan de seguridad).

La orquestación de los tres servicios se define en `docker-compose.yml`, con Nginx como proxy inverso y único servicio expuesto al exterior:

```
# docker-compose.yml (resumen)
services:
  db: { image: mariadb:11, volumes: [db_data:/var/lib/mysql] }
  api: { build: ./backend, expose: ["8000"] } # no publica puerto al exterior
  nginx: { image: nginx:alpine, ports: ["80:80", "443:443"] } # unico servicio expuesto

# Nginx: proxy inverso a la API
location / {
  proxy_pass http://api:8000;
  client_max_body_size 200M; # subidas de video grandes
  proxy_read_timeout 600s; # analisis prolongado
}
```

Un problema real afrontado durante el despliegue fue la incompatibilidad de la CPU del servidor con las instrucciones AVX que requieren TensorFlow y MediaPipe. El fallo se diagnosticó mediante el registro del núcleo (`dmesg`), que mostraba un «invalid opcode» al cargar las librerías:

```
traps: python3.10[1514] trap invalid opcode ip:7f01... in libtensorflow_framework.so.2
```

El problema se resolvió alojando la máquina virtual en un servidor cuyo procesador sí expone AVX.

5.9.3 *Despliegue del cliente*

El despliegue de la parte móvil es considerablemente más sencillo: se reduce a la instalación de la aplicación Android, empaquetada en un archivo `.apk` firmado, sobre el dispositivo del usuario. La distribución se realiza mediante instalación directa del APK, sin publicación en tiendas de aplicaciones, por tratarse de un prototipo académico. Su publicación en tiendas oficiales (como PlayStore) queda planteada como posible línea de trabajo futuro.

6 Limitaciones, líneas de trabajo futuras y conclusiones

6.1 Limitaciones

Aunque el resultado final del sistema es satisfactorio, a lo largo del desarrollo han aparecido diversas limitaciones que han condicionado el alcance y la precisión alcanzados y que han impedido conseguir un resultado aún mejor. Algunas ya se han mencionado a lo largo de la memoria como, por ejemplo, la escasez de vídeos de sentadillas con ejecuciones defectuosas, pero conviene recogerlas aquí de forma ordenada junto con otras que no se han tratado hasta ahora.

La limitación más determinante ha sido la relativa a los datos. El conjunto de entrenamiento se ha construido desde cero y, pese al esfuerzo invertido, sigue siendo reducido y está desequilibrado hacia las ejecuciones correctas, mucho más fáciles de obtener que las defectuosas. A ello se suma la subjetividad inherente al etiquetado manual, que se mitigó simplificando la escala de valoración a tres categorías, pero que no desaparece por completo. Esta limitación es la que mejor explica el desempeño más modesto de algunos indicadores, especialmente el ritmo, y de manera más acusada en el plano lateral.

Una limitación de carácter estructural es el uso de una única cámara. El sistema infiere la profundidad (la coordenada Z) a partir de una sola vista, por lo que esta es intrínsecamente menos precisa que las coordenadas del plano de la imagen, y ese error se propaga a todas las métricas tridimensionales. Como consecuencia directa, hay indicadores que no pueden calcularse de forma fiable según el plano de grabación: el valgo de rodilla y la simetría resultan inviables desde una vista lateral, mientras que la inclinación del torso, que se aprecia mejor desde una vista lateral, es más difícil de capturar cuando solo se dispone de una vista frontal, al depender de la profundidad. Se trata de una limitación propia del planteamiento del sistema, asumida de forma consciente a cambio de la accesibilidad que aporta trabajar con un único dispositivo.

Relacionada con la anterior está la variabilidad de las condiciones de grabación: los distintos encuadres, ángulos e iluminaciones con que un usuario puede grabarse. Aunque OpenCV y, sobre todo, la robustez de MediaPipe ofrecen muy buenos resultados frente a esta diversidad, sigue siendo una fuente de error; no obstante, deja de ser crítica en la medida en que el conjunto de datos cubra un número suficiente de escenarios posibles, lo que enlaza de nuevo con la dificultad de reunir suficientes datos.

El tiempo de desarrollo ha sido otra limitación evidente. Un proyecto de esta naturaleza, con una fuerte componente de investigación, se beneficia enormemente de la iteración, y con más tiempo habría sido posible recopilar más datos y refinar los modelos hasta obtener un rendimiento superior. Incluso poder haber reunido datos de otro tipo de ejercicios para haber contemplado algún ejercicio más además de la sentadilla.

A las limitaciones técnicas se añaden otras limitaciones de conocimiento. Por un lado, la falta de conocimientos previos en biomecánica, entrenamiento de fuerza y salud; una carencia que se ha intentado paliar mediante la consulta de fuentes especializadas, el apoyo de herramientas de inteligencia artificial e incluso la consulta a personas expertas, pero que, al tratarse del dominio sobre el que se construye todo el análisis, supone una limitación real. Por otro, la ausencia de experiencia previa con varias de las tecnologías empleadas: no se había desarrollado antes para Android ni trabajado con redes neuronales o modelos de inferencia, lo que ha hecho el trabajo considerablemente más exigente que si se hubieran empleado tecnologías ya conocidas. En este punto, tanto los repositorios y recursos disponibles en internet, los cuales están referenciados en la bibliografía, como las herramientas de inteligencia artificial han resultado de gran ayuda para el aprendizaje y, en el caso de estas últimas, también como apoyo durante la programación, especialmente en el desarrollo del frontend.

Además de la inexperiencia propia de un estudiante de grado, una limitación menos evidente pero igualmente importante ha sido la dificultad para dimensionar el propio proyecto. Al abordar tareas nunca antes realizadas, resultaba muy complicado anticipar el esfuerzo que requeriría cada una o hasta qué punto un objetivo era sencillo o complejo. Esta incertidumbre ha obligado a replanificar de forma continua y a reorientar objetivos sobre la marcha, como ocurrió al pasar de un único modelo a la arquitectura final de dos etapas, o el cambio de la idea inicial de poder soportar varios ejercicios a únicamente la sentadilla.

Por último, también el despliegue ha impuesto sus límites, como ya ha sido mencionado en el apartado Despliegue.

6.2 Líneas de trabajo futuras

Como se ha mencionado en el apartado anterior de las limitaciones, el sistema desarrollado no ha alcanzado la totalidad de lo que se planteó en un inicio, lo que abre varias líneas de trabajo futuro bastante claras.

El objetivo original era una aplicación completa para el usuario de gimnasio, capaz de analizar no solo la sentadilla, sino la mayoría de los ejercicios de fuerza. La principal línea futura es, por tanto, extender el sistema a otros ejercicios. Esto implica reentrenar la red BiLSTM con ejemplos de nuevos movimientos para que sea no solo capaz de segmentar las repeticiones de un abanico más amplio de ejercicios sino también reconocer que ejercicio se está realizando, además, habría que adaptar el motor de biomecánica para cada tipo de ejercicio y entrenar un modelo de evaluación específico para cada uno de ellos. La parte de la aplicación móvil sería la menos costosa: gracias a su diseño modular, al patrón MVVM y a la arquitectura limpia, ampliarla para soportar nuevos ejercicios resultaría rápido y sencillo. La verdadera dificultad sería una vez más, en construir un conjunto de datos por cada ejercicio que se desee añadir, suficiente para reentrenar la red y generar los nuevos modelos. En esta línea, sería muy valioso desarrollar herramientas que aceleren la creación de estos datasets, por ejemplo, mediante etiquetado semiautomático o ampliar las técnicas de aumentado de datos, más allá del espejo ya empleado, dado que ese es el principal cuello de botella del proyecto.

Un segundo grupo de mejoras tiene que ver con la precisión y la velocidad de la captura. Por un lado, convendría evaluar los modelos más ligeros de MediaPipe, midiendo su precisión y su tiempo de inferencia para determinar el punto óptimo entre ambos y agilizar así la respuesta de la aplicación; este análisis abriría además la puerta a una posible inferencia en el propio dispositivo e, idealmente, a un análisis en tiempo real además del análisis de vídeo ya grabado. Por otro lado, el uso de dos cámaras grabando simultáneamente desde ángulos distintos proporcionaría dos vistas del movimiento y, con ello, una precisión muy superior, al eliminar buena parte de las limitaciones derivadas de inferir la profundidad desde una sola vista. Una forma de incorporarlo sin complicar el uso habitual sería permitir subir dos vídeos de una misma sesión grabados desde dispositivos distintos; no obstante, es previsible que pocos usuarios recurran a esta opción, por la dificultad de disponer de dos dispositivos a la vez, salvo en escenarios concretos como grabar con la ayuda de otra persona. Dentro de esta misma línea se encuentra también la mejora de los indicadores, aunque el resultado no ha sido malo, sería posible mejorar sensiblemente la precisión con un conjunto mayor de datos, resultando todo esto en un sistema más rápido y preciso.

Otra mejora interesante, viable una vez se disponga de un volumen de datos suficiente, sería tener en cuenta la información del propio usuario como el objetivo

personal, el peso, la altura y el resto de datos recogidos en la encuesta inicial, para que tanto las recomendaciones como el comportamiento de la aplicación dejen de ser generalistas y se adapten al perfil concreto de cada persona.

Una vez que la aplicación cubriera un abanico amplio de ejercicios, constituiría ya un producto bastante completo, en el que no quedarían muchas funcionalidades de fondo por incorporar. Aun así, seguirían abiertas mejoras de carácter más incremental: la integración de un asistente conversacional (chatbot) que oriente al usuario, mejoras en la interfaz y en la calidad de las gráficas, o la exportación de informes por sesión individual, además de la global ya existente. Por último, y como paso natural para acercar el sistema a un uso real, quedaría su publicación en las tiendas de aplicaciones oficiales, como Google Play, lo cual a día de hoy ha sido descartado por tratarse de un prototipo académico.

6.3 Conclusión técnica

El objetivo principal planteado al inicio del trabajo —poner el análisis biomecánico del entrenamiento de fuerza al alcance de cualquier usuario, mediante un sistema de seguimiento completo de la sentadilla a través de un dispositivo móvil convencional— se ha alcanzado. El resultado es un sistema funcional y desplegado en producción que, a partir de un simple vídeo grabado con un teléfono Android, extrae la pose del usuario, evalúa los principales indicadores biomecánicos de cada repetición y devuelve una valoración comprensible, acompañada de recomendaciones y de un seguimiento de la evolución a lo largo del tiempo.

En cuanto a los subobjetivos funcionales, puede afirmarse que se han cumplido en su práctica totalidad. El pipeline de visión extrae las coordenadas tridimensionales de los puntos clave mediante MediaPipe BlazePose; el motor de biomecánica calcula las métricas articulares, de estabilidad y de ritmo aplicando filtrado, suavizado y normalización; la red BiLSTM segmenta las fases de la sentadilla y delimita las repeticiones; y el modelo XGBoost clasifica cada indicador en las categorías erróneo, aceptable y excelente, adaptándose además al plano de grabación detectado. Sobre esta base se ha construido un backend con FastAPI —con autenticación mediante JWT y procesamiento asíncrono— y una aplicación Android nativa, estructurada según los principios de la arquitectura limpia y el patrón MVVM, que integra el análisis técnico, el seguimiento longitudinal del usuario y el contenido educativo. Finalmente, el conjunto se

ha desplegado de forma contenedorizada con Docker y Nginx sobre HTTPS en un servidor de la Universidad, completando así el ciclo desde la captura del vídeo hasta la puesta en producción.

Desde un punto de vista técnico, la conclusión más relevante es que la decisión de separar el análisis en dos etapas —una red BiLSTM para la segmentación de las fases y un modelo XGBoost para la evaluación de los KPIs— resultó determinante para la viabilidad del sistema. Esta arquitectura permitió sortear el principal condicionante del proyecto, la escasez de datos, aprovechando la menor necesidad de ejemplos de los modelos basados en árboles. Igualmente decisiva fue la elección de un estimador de pose con salida tridimensional a partir de una única cámara, sin la cual buena parte del análisis biomecánico no habría sido posible. La evaluación de los modelos confirma, por su parte, un comportamiento coherente: el sistema es sólido en las tareas estructurales, como el conteo de repeticiones y la segmentación de fases, y más modesto en los indicadores dinámicos, más dependientes de la cantidad y variedad de datos disponibles.

En conjunto, el sistema cumple el propósito para el que fue concebido: ofrecer una herramienta de apoyo accesible para la valoración técnica de la sentadilla, consciente de sus propias limitaciones —detalladas en el apartado correspondiente— y con un recorrido de mejora claro, recogido en las líneas de trabajo futuro.

6.4 Conclusión personal

A pesar de las limitaciones y de las posibilidades de mejora futura, considero que el resultado es una aplicación completa y útil pero, sobre todo, y es lo más relevante en un Trabajo Fin de Grado, el reflejo de un proceso ordenado y completo que abarca todas las fases de un sistema real: desde la investigación inicial hasta el despliegue en producción, pasando por el diseño, la construcción del conjunto de datos, el entrenamiento de los modelos y su integración. Un proceso que, lejos de ser lineal, ha exigido adaptarse continuamente a los problemas, reorganizar el trabajo y replantear decisiones sobre la marcha.

En lo personal, estoy muy satisfecho de haber sido capaz de sacar adelante un proyecto de este calibre. Con él ha cobrado sentido buena parte de lo estudiado a lo largo de los cuatro años del grado de Ingeniería Informática. Asignatura a asignatura, uno tiene a veces la sensación de que los contenidos están desconectados y de que no terminan de prepararte para construir algo real por ti mismo; este trabajo me ha demostrado lo

contrario. No me ha dado todas las respuestas, pero sí las bases y, sobre todo, la forma de pensar necesaria para enfrentarme a problemas que nunca había abordado y resolverlos de manera autónoma e ingeniosa.

Precisamente ahí ha estado la mayor diferencia respecto a los ejercicios académicos a los que estaba acostumbrado. En una práctica de clase existe un enunciado acotado y una solución que se sabe alcanzable; aquí no había ni camino marcado ni garantía de que cada idea fuera a funcionar. He tenido que investigar, decidir y responsabilizarme de cada parte del sistema, asumiendo que muchas de esas decisiones surgían de la prueba y el error. El carácter investigador del proyecto, especialmente en su componente de inteligencia artificial, me ha obligado a convivir con la incertidumbre y a replanificar constantemente, algo que al principio resultaba incómodo y que con el tiempo he aprendido a gestionar.

A lo largo del desarrollo he tenido que superar obstáculos muy distintos entre sí. La construcción del conjunto de datos desde cero ha sido, con diferencia, lo más costoso y lo que más paciencia ha requerido. He aprendido prácticamente de cero tecnologías que nunca había utilizado, como el desarrollo en Android o el trabajo con redes neuronales y modelos de inferencia, apoyándome en la documentación, en recursos abiertos y en herramientas de inteligencia artificial. He tenido que adentrarme en un dominio ajeno a la informática, la biomecánica, buscando información y consultando a personas expertas. E incluso me he topado con problemas tan inesperados como una incompatibilidad de hardware en el servidor que estuvo a punto de bloquear el despliegue. Cada uno de estos imprevistos, por frustrante que resultara en su momento, ha terminado siendo una de las partes de las que más he aprendido.

Más allá de lo técnico, lo que me llevo de este trabajo son las competencias transversales: la capacidad de aprender por mi cuenta, de investigar con criterio, de organizar un proyecto largo bajo incertidumbre y de perseverar cuando las cosas no salen a la primera. Haber llevado un sistema desde una simple idea hasta algo desplegado y funcionando me ha dado una confianza que ningún ejercicio de clase me había proporcionado. Sin duda, ha sido el proyecto más ambicioso, completo y exigente de toda la carrera, y el que mejor me ha hecho sentir, por fin, lo que significa trabajar como ingeniero.

Lista de acrónimos

- **2D.** Dos dimensiones.
- **3D.** Tres dimensiones.
- **API.** Application Programming Interface (interfaz de programación de aplicaciones).
- **APK.** Android Package Kit (formato de paquete de las aplicaciones Android).
- **ASGI.** Asynchronous Server Gateway Interface.
- **AVX.** Advanced Vector Extensions (extensiones vectoriales del procesador).
- **BGR.** Blue-Green-Red (orden de canales de color que emplea OpenCV).
- **BiLSTM.** Bidirectional Long Short-Term Memory (red recurrente bidireccional).
- **CAFYD.** Ciencias de la Actividad Física y del Deporte.
- **CNN.** Convolutional Neural Network (red neuronal convolucional).
- **CPU.** Central Processing Unit (unidad central de procesamiento).
- **DTO.** Data Transfer Object (objeto de transferencia de datos).
- **FPS.** Frames Per Second (fotogramas por segundo).
- **GRU.** Gated Recurrent Unit.
- **HTTP / HTTPS.** HyperText Transfer Protocol (Secure).
- **IA.** Inteligencia Artificial.
- **JSON.** JavaScript Object Notation.
- **JWT.** JSON Web Token.
- **KPI.** Key Performance Indicator (indicador clave de rendimiento).
- **LSTM.** Long Short-Term Memory.
- **ML.** Machine Learning (aprendizaje automático).
- **MVP.** Minimum Viable Product (producto mínimo viable).
- **MVVM.** Model-View-ViewModel (patrón de arquitectura de interfaz).
- **NaN.** Not a Number (marca de valor numérico ausente).
- **ORM.** Object-Relational Mapping (mapeo objeto-relacional).
- **PDF.** Portable Document Format.
- **REST.** Representational State Transfer.
- **RGB.** Red-Green-Blue (espacio de color).
- **RNN.** Recurrent Neural Network (red neuronal recurrente).
- **RPE.** Rate of Perceived Exertion (esfuerzo percibido).
- **SQL.** Structured Query Language.

- **SSH.** Secure Shell.
- **ST-GCN.** Spatial Temporal Graph Convolutional Network.
- **TFG.** Trabajo Fin de Grado.
- **TLS.** Transport Layer Security.
- **UX.** User Experience (experiencia de usuario).
- **XGBoost.** Extreme Gradient Boosting (modelo de potenciación de gradiente sobre árboles).

Glosario

- **Aprendizaje profundo (deep learning).** Subcampo del aprendizaje automático basado en redes neuronales con múltiples capas que aprenden las características relevantes directamente de los datos.
- **Aumentado de datos (data augmentation).** Técnicas que generan nuevos ejemplos de entrenamiento a partir de los existentes (rotaciones, deformaciones, ruido), útiles cuando se dispone de pocos datos.
- **Biomecánica.** Disciplina que estudia el movimiento del cuerpo humano aplicando los principios de la mecánica.
- **Centro de sustentación.** Punto que representa la base de apoyo del cuerpo, calculado como el promedio de los puntos del pie; se utiliza como referencia de estabilidad.
- **Dropout.** Técnica de regularización que desactiva aleatoriamente neuronas durante el entrenamiento para reducir el sobreajuste.
- **Endpoint.** Punto de acceso de una API REST, identificado por una URL y un método HTTP, que expone una operación del sistema.
- **Estimación de pose.** Tarea de visión por computador que localiza los puntos anatómicos del cuerpo en una imagen o vídeo.
- **Fotograma (frame).** Cada una de las imágenes fijas que, mostradas en sucesión, componen un vídeo.
- **Inyección de dependencias.** Patrón que proporciona a un componente las instancias que necesita en lugar de que las cree él mismo, reduciendo el acoplamiento.
- **Keypoint** punto clave anatómico (articulación o referencia) detectado por la estimación de pose.
- **Sobreajuste (overfitting).** Situación en la que un modelo memoriza los datos de entrenamiento y pierde capacidad de generalizar a datos nuevos.
- **Parada temprana (early stopping).** Técnica que detiene el entrenamiento cuando el rendimiento sobre el conjunto de validación deja de mejorar.
- **Pipeline.** Secuencia encadenada de etapas de procesamiento por las que pasan los datos.
- **Proxy inverso.** Servidor intermedio que recibe las peticiones de los clientes y las redirige al servidor de aplicaciones, gestionando aspectos como la terminación TLS.
- **Filtro de Savitzky-Golay.** Filtro de suavizado que ajusta un polinomio local a la señal, atenuando el ruido sin deformar la forma del movimiento.

- **Serie temporal.** Secuencia de datos ordenados en el tiempo, en la que el orden es relevante.
- **Sentadilla.** Ejercicio de fuerza multiarticular que consiste en flexionar y extender de forma coordinada caderas, rodillas y tobillos.
- **Token.** Credencial digital que acredita la identidad del usuario en cada petición; en este sistema, en formato JWT.
- **Valgo de rodilla.** Desviación de la rodilla hacia el interior respecto a la línea que une la cadera con el tobillo durante el movimiento.
- **Validación cruzada.** Método de evaluación que divide los datos en varios pliegues para estimar el rendimiento de forma robusta; en este trabajo, agrupada por vídeo.
- **Worker.** Proceso del servidor que atiende peticiones; cada uno carga su propia copia de los modelos en memoria.

Bibliografía

Trabajos y artículos académicos consultados

- **[Pardo Feijoo]** Pardo Feijoo, S. Reconocimiento de objetos mediante el modelo Bag of Words. Proyecto Fin de Carrera, Universidad Carlos III de Madrid. <https://e-archivo.uc3m.es/entities/publication/0995cbae-6d33-45a9-ad3f-1ce456a97f24>
- **[García]** García, P. P. Clasificación de imágenes mediante momentos de Hu y redes neuronales. Trabajo Fin de Máster. <https://www.academia.edu/download/61283296/ProyectoFinMasterPedroPablo20191120-128137-13v2s7i.pdf>
- **[Santa María et al.]** Santa María et al. Sistema de visión artificial para la detección de regiones y patrones de color y forma. Revista de la Universidad Nacional de San Martín. <https://revistas.unsm.edu.pe/index.php/rcsi/article/view/131>
- **[Oliva Rodríguez]** Oliva Rodríguez. Detección de objetos mediante redes neuronales convolucionales sobre OpenCV. Trabajo Fin de Grado, Universitat Politècnica de València. <https://riunet.upv.es/server/api/core/bitstreams/6eb5e602-81cd-4911-8647-0d482f61ed71/content>
- **[Martínez Llamas]** Martínez Llamas, J. Identificación de ballenas mediante transfer learning sobre redes convolucionales. Trabajo Fin de Grado, Universidad Politécnica de Madrid. <https://oa.upm.es/53050/>
- **[Lesiones lumbares]** Sistema de alerta para la prevención de lesiones lumbares en usuarios de gimnasio durante el peso muerto. Trabajo Fin de Grado, Universidad El Bosque. <https://repositorio.unbosque.edu.co/items/311b6cf2-0174-4166-bbd4-58474c5cc6f1>

Modelos, técnicas y artículos de referencia

- **[Bazarevsky et al., 2020]** Bazarevsky, V. et al. BlazePose: On-device Real-time Body Pose Tracking. arXiv:2006.10204.
- **[Lugaresi et al., 2019]** Lugaresi, C. et al. MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.
- **[Chen y Guestrin, 2016]** Chen, T. y Guestrin, C. XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD (KDD '16).
- **[Hochreiter y Schmidhuber, 1997]** Hochreiter, S. y Schmidhuber, J. Long Short-Term Memory. Neural Computation, 9(8).

- **[Schuster y Paliwal, 1997]** Schuster, M. y Paliwal, K. Bidirectional Recurrent Neural Networks. IEEE Transactions on Signal Processing, 45(11).
- **[Vaswani et al., 2017]** Vaswani, A. et al. Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).
- **[Yan et al., 2018]** Yan, S., Xiong, Y. y Lin, D. Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition. AAAI.
- **[Savitzky y Golay, 1964]** Savitzky, A. y Golay, M. J. E. Smoothing and Differentiation of Data by Simplified Least Squares Procedures. Analytical Chemistry, 36(8).
- **[Bradski, 2000]** Bradski, G. The OpenCV Library. Dr. Dobbs's Journal of Software Tools.
- **[Strömbäck et al., 2020]** Strömbäck, D., Huang, S. y Radu, V. MM-Fit: Multimodal Deep Learning for Automatic Exercise Logging across Sensing Devices. (Conjunto de datos MM-Fit).

Ciencias del deporte y escalas de valoración

- **[Gabbett, 2016]** Gabbett, T. J. The training-injury prevention paradox: should athletes be training smarter and harder? British Journal of Sports Medicine, 50(5).
- **[Borg, 1982]** Borg, G. A. V. Psychophysical bases of perceived exertion. Medicine & Science in Sports & Exercise, 14(5).
- **[Huskisson, 1974]** Huskisson, E. C. Measurement of pain. The Lancet, 304(7889). (Escala Visual Analógica del dolor, EVA).

Repositorios y proyectos de código

- **[Riccio]** Riccio, R. Fitness AI Trainer with Automatic Exercise Recognition and Counting. GitHub. <https://github.com/RiccardoRiccio/Fitness-AI-Trainer-With-Automatic-Exercise-Recognition-and-Counting> (artículo asociado: arXiv:2411.11548).
- **[geaxgx]** geaxgx. depthai_blazepose. GitHub. https://github.com/geaxgx/depthai_blazepose
- **[cv4Africa]** Ro-ya-cv4Africa. acvdatasets. GitHub. <https://github.com/Ro-ya-cv4Africa/acvdatasets>
- **[Fitness Classification]** Real-Time Fitness Exercise Classification and Counting from Video Frames. ResearchGate, publicación 385920370.

Conjuntos de datos y fuentes de vídeo

- **[Pexels]** Pexels. Banco de vídeos libres de derechos. <https://www.pexels.com/es-es/buscar/videos/squat/>
- **[iStock]** iStock. Banco de imágenes y vídeos. <https://www.istockphoto.com>
- **[Zenodo]** Conjuntos de datos de ejercicios en Zenodo. <https://zenodo.org/records/13305826> y <https://zenodo.org/records/7672767>
- **[Kaggle]** Kaggle. Plataforma de conjuntos de datos. <https://www.kaggle.com>
- **[MM-Fit]** MM-Fit dataset. <https://mmfit.github.io/>

Documentación de herramientas y frameworks

- **[OpenCV]** OpenCV. Documentación oficial. <https://docs.opencv.org>
- **[MediaPipe]** Google. MediaPipe (BlazePose). <https://developers.google.com/mediapipe>
- **[TensorFlow]** TensorFlow. Documentación oficial. <https://www.tensorflow.org>
- **[Keras]** Keras. Documentación oficial. <https://keras.io>
- **[XGBoost]** XGBoost. Documentación oficial. <https://xgboost.readthedocs.io>
- **[scikit-learn]** scikit-learn. Documentación oficial. <https://scikit-learn.org>
- **[FastAPI]** Ramírez, S. FastAPI. Documentación oficial. <https://fastapi.tiangolo.com>
- **[SQLAlchemy]** SQLAlchemy. Documentación oficial. <https://www.sqlalchemy.org>
- **[Android]** Android Developers (Jetpack Compose, Hilt, Retrofit, WorkManager, DataStore). <https://developer.android.com>
- **[Docker]** Docker. Documentación oficial. <https://docs.docker.com>
- **[Nginx]** Nginx. Documentación oficial. <https://nginx.org>

Ingeniería del software y estimación

- **[Karner, 1993]** Karner, G. *Resource Estimation for Objectory Projects*. Objective Systems SF AB, 1993.
- **[Cohn, 2005]** Cohn, M. *Agile Estimating and Planning*. Prentice Hall, 2005.
- **[Brown]** Brown, S. *The C4 model for visualising software architecture*. <https://c4model.com>
- **[Gamma et al., 1995]** Gamma, E., Helm, R., Johnson, R. y Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- **[Martin, 2017]** Martin, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

Normativa, estándares y seguridad

- **[RGPD]** Reglamento (UE) 2016/679 (RGPD). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- **[LOPDGDD]** Ley Orgánica 3/2018 (LOPDGDD). <https://www.boe.es/eli/es/lo/2018/12/05/3>
- **[LO 1/1982]** Ley Orgánica 1/1982, de protección civil del derecho al honor, a la intimidad personal y familiar y a la propia imagen. <https://www.boe.es/eli/es/lo/1982/05/05/1>
- **[Reglamento IA]** Reglamento (UE) 2024/1689 (Reglamento de Inteligencia Artificial). <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- **[AEPD]** AEPD. *Gestión del riesgo y evaluación de impacto en tratamientos de datos personales*.
- **[Magerit]** CCN. *Magerit v3 — Metodología de Análisis y Gestión de Riesgos de los Sistemas de Información*.
- **[OWASP]** OWASP Foundation. *OWASP Top 10 y Application Security Verification Standard (ASVS)*.